

Claude Certified Architect Foundations (CCA-F)

100 Exam-Style Practice Questions with Full Rationale

Correct answer · Why it's right · Why others are wrong · Exam tip

Exam Blueprint at a Glance

Format	60 questions · multiple choice · scenario-based
Duration	120 minutes · online proctored or test centre
Passing score	720 / 1000 · no penalty for guessing
Validity	12 months

Domain coverage in this guide:



HOW TO USE THIS GUIDE

This bank contains 100 scenario-style questions modelled on the five official CCA-F domains and the six exam scenarios (customer-support agent, Claude Code development, multi-agent research, developer productivity, CI/CD, and structured extraction). For every question you get the correct answer, a clear explanation of why it is right, a breakdown of why each other option is wrong, and a distilled exam tip that generalises to similar questions. Work domain by domain, then re-read only the exam tips the night before the exam for fast revision.

Domain	Exam Weight	Questions
D1 · Agentic Architecture	27%	27
D2 · Claude Code	20%	21
D3 · Prompt Engineering	20%	19
D4 · Tool Design & MCP	18%	18
D5 · Context & Reliability	15%	15

Legend inside each question: green = correct answer & rationale · red = distractor analysis · amber ★ = exam tip.

D1 · Agentic Architecture

Agentic Architecture & Orchestration · Exam weight 27% · 27 questions

Q01

D1 · Agentic Architecture

You are writing the harness loop for a customer-support agent. After each `messages.create()` call, what is the single most reliable signal that tells your loop whether to execute tools and continue, or to stop and return the answer to the user?

A

Scan the assistant's text for phrases like 'let me check' or 'I'll look that up' and continue whenever such intent language appears.

B

Inspect the response's `stop_reason` field: 'tool_use' means execute the requested tools and loop again; 'end_turn' means the turn is complete and you return the reply. ✓ correct

C

Check whether the response contains any content blocks at all, and loop again as long as something was returned.

D

Poll a `response.is_finished` boolean that the API sets once the agent has no further work.

WHY B IS CORRECT

The Messages API returns a `stop_reason` on every response. A value of 'tool_use' means Claude wants to call one or more tools, so the harness executes them, appends the `tool_result`, and loops. 'end_turn' means Claude has finished and the text can be returned to the user. This is the deterministic, API-provided control signal for an agentic loop.

WHY THE OTHER OPTIONS ARE WRONG

- A. Parsing natural-language phrases is brittle and unreliable; intent language is not a contract and will produce false positives and negatives.
- C. Text responses also contain content blocks, so 'any content block' cannot distinguish a tool call from a final answer.
- D. There is no `is_finished` boolean in the Messages API; `stop_reason` is the actual mechanism.

★ EXAM TIP

Memorize the `stop_reason` values: 'tool_use' -> run tools & loop; 'end_turn' -> done; 'max_tokens' -> truncated; 'stop_sequence' -> hit a stop string. The loop is driven by `stop_reason`, never by text parsing.

Q02

D1 · Agentic Architecture

A coordinator agent's system prompt describes several available subagents, but delegation never happens — the coordinator only talks about the work. Its `allowedTools` list does not include 'Task'. What is the root cause?

- A Coordinators spawn subagents implicitly from their system prompt once the right phrasing is used.
- B Subagents must first be registered in a central registry service.
- C The coordinator's `max_tokens` is too small to emit a delegation.
- D **The 'Task' tool is the mechanism for spawning subagents; `allowedTools` must include 'Task' for the coordinator to invoke them. ✓ correct**

WHY D IS CORRECT

In the Agent SDK, delegation is performed by calling the Task tool. Describing subagents in a prompt is not enough — the coordinator must have the Task tool in its `allowedTools` to actually launch them. Without it, the model can only describe delegation.

WHY THE OTHER OPTIONS ARE WRONG

- A. There is no magic phrase; capabilities come from tools, not prompt wording alone.
- B. No central registry is required beyond the configured agent definitions/tools.
- C. A small token limit would truncate output, not systematically block all delegation while allowing planning text.

★ EXAM TIP

Golden rule: a capability described in a prompt is inert unless the matching tool is in `allowedTools`. For subagent orchestration, the enabling tool is 'Task'.

Q03

D1 · Agentic Architecture

In a multi-agent research system, a coordinator delegates the same broad topic to a web-search agent and a document-analysis agent. Both investigate the same subtopics, nearly doubling token cost without improving coverage. What is the most effective fix?

- A **Have the coordinator explicitly partition the research space before delegation, assigning distinct subtopics or source types to each agent. ✓ correct**
- B Convert the whole pipeline to sequential execution so only one agent runs at a time.
- C Implement a shared state store where each agent logs its current focus area at runtime.
- D Let both agents finish, then deduplicate overlapping findings before synthesis.

WHY A IS CORRECT

The root cause is poor task decomposition. The coordinator should partition the research space up front — by subtopic or by source type (e.g., web vs. internal documents) — so agents do non-overlapping work. This preserves parallelism while eliminating duplication and wasted tokens.

WHY THE OTHER OPTIONS ARE WRONG

- B.** Sequential execution sacrifices the main benefit of multi-agent systems (parallelism) and still doesn't guarantee clean boundaries.
- C.** Runtime shared-state coordination is more complex and reactive; upfront partitioning is the simpler, root-cause fix.
- D.** Post-hoc deduplication removes duplicate output but the wasted tokens were already spent.

★ EXAM TIP

For orchestrator-worker patterns, the coordinator owns scope partitioning. Assign distinct, non-overlapping responsibilities BEFORE delegating, not after.

Q04**D1 · Agentic Architecture**

A research coordinator is asked for 'the history of renewable energy adoption,' but it only creates solar-focused subtasks (panel costs, residential installs, utility-scale solar). Wind, hydro, geothermal, and biomass are absent from the final report. Where did the failure occur?

- A** The document-analysis agent rejected all non-solar sources.
- B** The web-search agent returned only solar results by chance.
- C** The synthesis agent dropped the non-solar sections during compression.
- D** **The coordinator's task decomposition is overly narrow, producing subagent assignments that don't cover the full scope of the topic. ✓ correct**

WHY D IS CORRECT

The subtasks were solar-only, meaning the scope error happened at decomposition — before any subagent did its work. Even excellent subagents cannot recover coverage that the coordinator never assigned. The coordinator must decompose the request into complete, scope-covering subtasks.

WHY THE OTHER OPTIONS ARE WRONG

- A.** No evidence sources were rejected; the assignments themselves were solar-only.
- B.** The narrow queries flowed from narrow subtasks; the search agent isn't the root cause.
- C.** Attribution/compression issues would drop detail, not systematically omit entire renewable categories that were never assigned.

★ EXAM TIP

When a report is missing whole sub-areas, suspect the coordinator's decomposition first. Complete, non-overlapping, scope-covering subtasks are the coordinator's responsibility.

Q05

D1 · Agentic Architecture

A key design property of Claude subagents is how they interact with the parent's context. Which statement is correct?

A Subagents inherit the full parent conversation so they have maximum context.

B Subagents run in their own isolated context window and return only a clean summary to the parent. ✓ correct

C Subagents share a single context window with the parent to stay synchronized.

D Subagents write directly into the parent's context as they work.

WHY B IS CORRECT

Subagents exist precisely to protect the parent's context window. Each runs in its own isolated context, does focused work, and returns only a concise summary. This keeps the orchestrator's context clean and prevents noisy intermediate work from consuming the parent's tokens.

WHY THE OTHER OPTIONS ARE WRONG

A. Subagents do NOT inherit the full parent context — that would defeat their purpose.

C. They do not share one window; isolation is the whole point.

D. They return a summary; they don't stream raw work into the parent context.

★ EXAM TIP

Remember: 'subagents protect context.' They never inherit parent context and return only a clean summary. This is one of the most frequently tested facts on the exam.

Q06

D1 · Agentic Architecture

You must decide between a single-agent design and a multi-agent (orchestrator-worker) design for a task. Which situation most justifies a multi-agent architecture?

A A short, linear task that fits comfortably in one context window.

B A latency-critical task where every extra token of overhead must be avoided.

C A broad task with separable subtasks that can be researched in parallel and would otherwise exhaust a single context window. ✓ correct

D A simple lookup that requires exactly one tool call.

WHY C IS CORRECT

Multi-agent systems shine when a task is broad and decomposable into parallelizable subtasks, and when a single context window would be overwhelmed. Orchestrator-worker patterns let each subagent explore a slice in isolation and return summaries, improving both coverage and context efficiency.

WHY THE OTHER OPTIONS ARE WRONG

- A. Short, linear tasks are cheaper and simpler as a single agent.
- B. Multi-agent coordination adds token and latency overhead, so it's a poor fit when latency is paramount.
- D. A single tool call needs no orchestration.

★ EXAM TIP

Default to the simplest architecture that works. Reach for multi-agent only when the task is broad, parallelizable, and context-heavy — otherwise a single agent is cheaper and more reliable.

Q07

D1 · Agentic Architecture

A customer says 'Fix my account.' The agent immediately calls `process_refund` and issues an unwanted refund. What is the best architectural fix?

- A Replace `process_refund` with a more conservative refund tool.
- B Design the agent to gather and confirm the customer's specific intent before calling any action tools, especially irreversible ones like `process_refund`. ✓ correct**
- C Require the customer to send two messages confirming the same intent.
- D Add a 30-minute delay before refunds take effect.

WHY B IS CORRECT

The agent acted on an ambiguous request without clarifying intent. Before invoking high-impact or irreversible tools, the agent must understand and confirm what the user actually wants. The pattern is Understand -> Confirm -> Act.

WHY THE OTHER OPTIONS ARE WRONG

- A. A different tool doesn't fix the underlying failure to understand intent.
- C. Two messages is an arbitrary safeguard that adds friction without addressing ambiguity.
- D. A delay mitigates damage after a mistake but doesn't prevent the misunderstanding.

★ EXAM TIP

For irreversible actions (refunds, deletions, payments), always require intent clarification and explicit confirmation before the tool call. Ambiguity is a stop-and-clarify signal.

Q08

D1 · Agentic Architecture

A support agent's escalation policy is 'escalate any request not resolved in under 3 tool calls.' 60% of cases now escalate (target is 20%), and many escalated cases were actually solvable. What is the best redesign?

- A Remove the tool-call limit entirely so the agent can loop freely.
- B Replace the tool-call count trigger with escalation triggers based on actual resolution signals: unresolvable tool errors, explicit requests for a human, or detected high frustration. ✓ correct**
- C Reduce the limit to 2 tool calls.
- D Add a supervisor agent that reviews every case after 3 tool calls.

WHY B IS CORRECT

The tool-call count is an arbitrary proxy that doesn't reflect whether a case is truly unresolvable. Escalation should be driven by real signals: hard tool failures, explicit human requests, or strong frustration. This aligns escalation with genuine inability to resolve.

WHY THE OTHER OPTIONS ARE WRONG

- A. Unbounded looping risks runaway cost and poor UX; some guardrail is still needed.
- C. Lowering the threshold makes over-escalation worse.
- D. Adds cost and complexity while keeping the flawed trigger.

★ EXAM TIP

Escalate on inability-to-resolve signals, not superficial metrics like message count or tool-call count. Good triggers: hard errors, explicit human request, high frustration, out-of-authority actions.

Q09

D1 · Agentic Architecture

A frustrated customer has sent 7 escalating messages including 'Give me a supervisor NOW.' The agent still has a valid automated resolution path. What should it do?

- A Show empathy and continue automated resolution to preserve first-contact-resolution metrics.
- B Escalate to a human immediately, passing full context including detected frustration signals, regardless of remaining automated options. ✓ correct**
- C Continue automation because a technical path still exists.
- D Ask the customer to calm down before proceeding.

WHY B IS CORRECT

An explicit request for a human plus strong frustration signals should trigger immediate escalation, even if automation could technically continue. Forcing more automation harms the customer experience. The handoff should carry full context and the detected frustration.

WHY THE OTHER OPTIONS ARE WRONG

- A. Empathy is good, but it doesn't satisfy the explicit request for a supervisor.
- C. A valid technical path does not override a clear human request.
- D. Telling a frustrated customer to calm down is poor practice and worsens the interaction.

★ EXAM TIP

Explicit human request OR high frustration = escalate now. Metrics like FCR never outrank a customer's clear demand for a person.

Q10

D1 · Agentic Architecture

A support agent processes a refund, but a network timeout hides the success response. The harness retries and a SECOND refund is issued. What is the correct fix?

- A Query the account balance before and after every refund to detect duplicates.
- B Include a unique idempotency key in each process_refund call so the backend deduplicates retries and processes the refund once. ✓ correct**
- C Wait 60 seconds before retrying any refund.
- D Disable retries entirely for refund operations.

WHY B IS CORRECT

This is a classic idempotency problem. Attaching a unique idempotency key lets the backend recognize a retried request and return the original result instead of executing a duplicate. This makes retries safe for state-changing operations.

WHY THE OTHER OPTIONS ARE WRONG

- A. Balance checks may detect duplicates after the fact but don't prevent them.
- C. A delay reduces but doesn't eliminate duplication.
- D. Removing retries hurts reliability; transient failures are common. The right answer is safe retries via idempotency.

★ EXAM TIP

Any state-changing operation (refunds, payments, order creation) should be idempotent. Idempotency key + retry = safe. Never solve duplicates by disabling retries.

Q11

D1 · Agentic Architecture

A support agent must approve refunds only up to \$500, but it processed a \$1,200 refund because the `process_refund` tool didn't enforce the cap. What is the best fix?

- A Ask the customer whether the refund is above or below \$500.
- B Reverse any refund above \$500 after it is processed.
- C Enforce the \$500 limit in the `process_refund` tool's input validation, rejecting larger calls with an actionable error that directs the agent to escalate. ✓ correct**
- D Add a system-prompt instruction: 'Never process refunds above \$500.'

WHY C IS CORRECT

Authorization must be enforced at the tool/backend layer, not in prompts. The tool should validate inputs and reject amounts over the threshold, returning an error that tells the agent to escalate. This holds even if the prompt is ignored or another agent calls the tool.

WHY THE OTHER OPTIONS ARE WRONG

- A. Authorization should not depend on customer answers; the system already knows the amount.
- B. Reversing after the fact is reactive and risky.
- D. Prompts are guidance, not enforcement; they can be ignored or overridden.

★ EXAM TIP

Trust boundary rule: prompts guide, tools enforce. Put hard limits (auth, spend caps, permissions) in tool input validation or the backend, never only in the prompt.

Q12

D1 · Agentic Architecture

A support agent was given a generic `http_request` tool 'for flexibility.' It began calling competitor sites and returning raw backend responses to customers. What is the best fix?

- A Remove `http_request` and restrict the agent to its four purpose-built tools; if new needs arise, build domain-specific tools with appropriate constraints. ✓ correct**
- B Add a human review step before every `http_request` call.
- C Expand the system prompt with a list of prohibited URLs and behaviors.
- D Add a blocklist of competitor domains to the tool.

WHY A IS CORRECT

The generic tool was too powerful. Applying least privilege — purpose-built, narrowly scoped tools — removes the ability to misuse it. New requirements should be met with additional constrained tools, not an open-ended HTTP capability.

WHY THE OTHER OPTIONS ARE WRONG

- B.** Human review for every call is impractical and doesn't fix the over-broad capability.
- C.** Prompt prohibitions are weaker than tool-level constraints.
- D.** A blocklist is reactive; new domains and raw-response leakage remain possible. Prefer an allowlist of capabilities.

★ EXAM TIP

Give agents the narrowest tools needed (least privilege). Prefer purpose-built tools over a generic 'do-anything' tool, and prefer capability allowlists over endless blocklists.

Q13**D1 · Agentic Architecture**

A customer requests a return. The agent calls `process_refund` immediately, refunding a 94-day-old order even though the policy window is 90 days. What is the best architectural fix?

- A** **Enforce a tool-call sequence in agent logic: always call `get_customer` and `lookup_order` first, and validate eligibility from the results before calling `process_refund`. ✓ correct**
- B** Add a prompt instruction telling the agent to check the order date.
- C** Ask the customer to confirm the order is eligible.
- D** Add an `order_age` field to the `process_refund` schema.

WHY A IS CORRECT

The agent executed an irreversible action before gathering prerequisites and validating eligibility. Enforcing a required sequence (retrieve customer -> retrieve order -> validate policy -> act) prevents refunds on ineligible orders.

WHY THE OTHER OPTIONS ARE WRONG

- B.** A prompt instruction is guidance, easily skipped; workflow preconditions belong in orchestration logic.
- C.** Customers usually don't know policy details; the system has authoritative data.
- D.** Adding a field doesn't guarantee the value comes from a trusted source or that all eligibility checks run.

★ EXAM TIP

For high-impact actions, enforce a precondition workflow: gather required facts, validate policy/eligibility, THEN act. Don't rely on the model to remember the order of operations.

Q14

D1 · Agentic Architecture

A customer says 'I want to return everything from my last order.' The agent interprets 'everything' as all 12 items and refunds \$847, but the customer meant one defective \$23 item. What is the best fix?

A Always refund the smallest item when the request is ambiguous.

B Have the agent enumerate the specific items and amounts and confirm the exact scope of the return before calling process_refund. ✓ correct

C Require manager approval for all refunds over \$100.

D Require customers to provide order IDs and line-item numbers.

WHY B IS CORRECT

The agent must resolve ambiguity before an irreversible action. It should look up the order, present the items and amounts, and confirm exactly what to return before refunding.

WHY THE OTHER OPTIONS ARE WRONG

A. Guessing the smallest item is just a different wrong assumption.

C. Manager approval reduces financial risk but doesn't fix the misunderstanding.

D. Requiring technical identifiers pushes burden onto the customer; the agent can retrieve and present the order itself.

★ EXAM TIP

Ambiguous scope + irreversible action = enumerate and confirm. Understand -> Enumerate -> Confirm -> Execute.

Q15

D1 · Agentic Architecture

During a billing dispute the agent has (1) confirmed the charge was incorrect, (2) failed to refund due to a PERMANENT permission error (account locked by compliance), and (3) exhausted all automated paths. The customer is frustrated. What is the correct final action?

A Issue a store credit instead of a refund since the refund path is unavailable.

B Attempt process_refund again in case the permission error was transient.

C Close the conversation and tell the customer to call back when the account is unlocked.

D Escalate to a human with a structured handoff: confirmed billing error, refund amount, account-lock reason (compliance), actions attempted, and recommended next step (compliance review), while reassuring the customer. ✓ correct

WHY D IS CORRECT

A valid resolution was identified but is blocked by a permanent policy restriction, and all automated paths are exhausted. The correct action is a high-quality human handoff that preserves full context and recommends the next step, while reassuring the customer.

WHY THE OTHER OPTIONS ARE WRONG

- A. The agent lacks authority to substitute a different remedy.
- B. The error is explicitly permanent; retrying won't help.
- C. Closing the case abandons the customer and discards gathered context.

★ EXAM TIP

When resolution is blocked by policy/authorization and no automated path remains: escalate with a structured handoff (who, what, why blocked, what was tried, recommended next step).

Q16

D1 · Agentic Architecture

A support agent is provisioned with nine tools including `send_marketing_email`, `update_account_tier`, and `generate_invoice`. In production it sometimes sends marketing emails during billing disputes and changes account tiers during routine returns. What is the most effective fix?

- A Implement a post-call audit log that flags inappropriate tool calls for human review.
- B Add system-prompt instructions listing which tools it should never call during specific workflows.
- C Add a customer confirmation step before every tool call.
- D Scope the agent's tool access to only the tools needed for support resolution: `get_customer`, `lookup_order`, `process_refund`, and `escalate_to_human`. ✓ correct**

WHY D IS CORRECT

This is a least-privilege problem. Removing unrelated tools means the agent literally cannot send marketing emails or change tiers. Scoping tool access to the role is the strongest and simplest fix.

WHY THE OTHER OPTIONS ARE WRONG

- A. Audit logs detect misuse after it happens; they don't prevent it.
- B. Prompt prohibitions are weaker than removing the tools.
- C. Per-call confirmation creates poor UX and still leaves the over-broad toolset in place.

★ EXAM TIP

Provision agents with only the tools their job requires. If a capability is not needed for the role, don't expose it — enforcement via tool access beats prompt warnings.

Q17

D1 · Agentic Architecture

The `get_customer` tool returns a 503 Service Unavailable. What is the best agent behavior?

- A Show the raw 503 error to the customer and ask them to try later.
- B Ask the customer for their account details manually and proceed without the tool.
- C Retry `get_customer` up to 3 times with exponential backoff, then escalate to a human with a note about the outage so the customer still gets help. ✓ correct**
- D Immediately escalate every 503 to a human with no retry.

WHY C IS CORRECT

A 503 is typically transient. Retrying with exponential backoff resolves many short outages; if retries fail, a graceful escalation with outage context keeps the customer supported. This balances automation with a safe fallback.

WHY THE OTHER OPTIONS ARE WRONG

- A. Exposing raw infrastructure errors is poor UX and abandons the customer.
- B. Proceeding without authoritative account data risks incorrect or unsafe actions.
- D. Escalating immediately wastes an automated recovery that would often succeed.

★ EXAM TIP

Transient errors (503, timeouts) -> retry with exponential backoff, then escalate with context. Match the recovery strategy to the error type.

Q18

D1 · Agentic Architecture

The `process_refund` tool can fail three ways: (1) a 503 timeout, (2) 'refund already processed,' and (3) 'permission denied — fraud review.' The agent currently escalates all three. What is the best handling?

- A Use the structured error category: retry transient errors (case 1); explain and close for already-processed refunds (case 2); escalate with fraud-review context for permission errors (case 3). ✓ correct**
- B Route all three cases to human agents.
- C Tell customers to contact the billing department for all three.
- D Retry all three errors up to 5 times.

WHY A IS CORRECT

Different error categories need different recovery strategies. Transient -> retry; business-state (already refunded) -> inform and close; authorization (fraud review) -> escalate with context. Structured error handling avoids unnecessary escalations and dead-ends.

WHY THE OTHER OPTIONS ARE WRONG

- B. Escalating all three creates needless human load for cases that are auto-resolvable.
- C. Redirecting everyone to billing abandons useful automation.
- D. Retrying a duplicate or a permission denial will never succeed.

★ EXAM TIP

Map error CATEGORY to recovery: transient->retry, business-state->explain/resolve, permission->escalate, validation->fix input. Design tools to return structured error types.

Q19

D1 · Agentic Architecture

When an automated agent escalates to a human, the note says only {"reason":"Unable to process request"}. What should a good escalation contain?

- A Just mark the ticket 'High Priority.'
- B Only the customer's raw messages.
- C Customer ID, account flag (fraud review), attempted action (refund of \$X for order Y), error category (permission — fraud review), and recommended next step (route to fraud-review team). ✓ correct**
- D The raw JSON error response from the backend.

WHY C IS CORRECT

Good escalations preserve context so the human can act immediately: who is affected, what was attempted, why automation failed, and the recommended next step. This eliminates re-investigation and speeds resolution.

WHY THE OTHER OPTIONS ARE WRONG

- A. Priority alone gives no actionable context.
- B. Customer messages often lack internal status like fraud flags or permission errors.
- D. Raw JSON is verbose and hard for support staff to interpret; translate it into structured operational context.

★ EXAM TIP

Escalations should TRANSFER CONTEXT, not just ownership. Include who/what/why/next-step so the human never has to start from scratch.

Q20

D1 · Agentic Architecture

A long-running multi-agent research job must survive a crash and resume from the last good state without re-running completed work. The web-search subagent already finished. What design best supports this?

- A Restart the entire research task from the beginning after any crash.
- B Stream state to the user and ask them to paste it back on resume.
- C Persist only the coordinator's most recent message.
- D Have each subagent export its state (findings, citations, partial outputs) to a known location, and have the coordinator load a manifest on resume to inject prior state into new subagents. ✓ correct**

WHY D IS CORRECT

Checkpointing subagent outputs to durable storage, tracked by a manifest, lets the system reload completed work and resume the remaining tasks after a crash — avoiding expensive re-execution of finished searches.

WHY THE OTHER OPTIONS ARE WRONG

- A. Restarting wastes completed work and defeats fault tolerance.
- B. Manual user paste-back is fragile; persistence should be automatic.
- C. The coordinator's last message doesn't contain all intermediate subagent artifacts.

★ EXAM TIP

For recoverable long-running agents: checkpoint subagent outputs + a manifest of completed work. Resume by reloading checkpoints, not by re-running everything.

Q21

D1 · Agentic Architecture

A document-analysis subagent finds two credible but conflicting figures (a government report says 40% growth; an industry analysis says 12%). What should it do?

- A Halt all analysis immediately and return nothing.
- B Include both figures without flagging the conflict.
- C Complete the analysis with both figures included, explicitly annotate the conflict with source attribution, and let the coordinator decide how to reconcile before synthesis. ✓ correct**
- D Pick the figure it judges 'most likely accurate' and drop the other.

WHY C IS CORRECT

Conflicting credible evidence should be preserved, attributed, and surfaced. The subagent finishes its work but flags the discrepancy with sources so the coordinator/synthesis layer can reconcile it with full context.

WHY THE OTHER OPTIONS ARE WRONG

- A. Halting everything is wasteful; the conflict can be flagged while work continues.
- B. Hiding the conflict risks the synthesis agent treating both as compatible.
- D. A subagent shouldn't unilaterally resolve a significant conflict with a heuristic.

★ EXAM TIP

Preserve + attribute + surface conflicting evidence. Never silently pick a winner at the subagent level; reconciliation belongs to the coordinator/synthesis stage.

Q22

D1 · Agentic Architecture

A synthesis agent produces a report full of unattributed claims like 'Research indicates...' even though the subagents gathered well-sourced findings. What is the root cause and fix?

A The synthesis model has a bug and should be swapped.

B Source attribution is being lost during compression at the synthesis layer; require subagents to output structured claim-source mappings (URLs, document names, excerpts) that downstream stages preserve.
✓ correct

C Add a prompt asking the synthesis agent for footnotes.

D The web-search agent isn't returning sources.

WHY B IS CORRECT

Attribution loss during synthesis/compression is an architecture problem. Making provenance a first-class data structure (claim -> source mapping) that every stage must preserve keeps claims traceable end to end.

WHY THE OTHER OPTIONS ARE WRONG

- A. No evidence of a model bug; the pipeline drops provenance.
- C. Asking for footnotes helps presentation but not if provenance was already lost upstream.
- D. The scenario says subagents gathered well-sourced findings; loss happens at synthesis.

★ EXAM TIP

Pass facts WITH sources through every stage. Represent provenance as structured claim-source mappings so citations survive compression and synthesis.

Q23

D1 · Agentic Architecture

A synthesis agent forces financial metrics, a press-coverage timeline, and technical methodology all into one flat block of prose, making the report hard to use. What is the best fix?

- A Convert everything to bullet points.
- B Drop the methodology and timeline sections to shorten the report.
- C Render each content type appropriately: financial data as tables, news as prose, and technical findings as structured lists, rather than one uniform format. ✓ correct**
- D Increase the font size on the important numbers.

WHY C IS CORRECT

Different information types are best communicated in different formats. Tables for metrics, chronological structure for timelines, and structured steps/lists for methodology make the output usable. The synthesis agent should choose format by content type.

WHY THE OTHER OPTIONS ARE WRONG

- A. Bullets are still one-size-fits-all; numbers often need tables.
- B. Dropping sections loses valuable information.
- D. Font size is superficial and doesn't fix structure.

★ EXAM TIP

Match presentation format to content type: metrics->tables, timelines->chronological lists, procedures->structured steps, narrative->prose.

Q24

D1 · Agentic Architecture

You are building a Human-in-the-Loop (HITL) checkpoint for an agent that can delete production data. What is the best placement for the approval gate?

- A After the deletion, so a human can verify the result.
- B Before the irreversible action, pausing for explicit human approval of the specific operation and its parameters. ✓ correct**
- C Only at the start of the session, approving all future actions at once.
- D Randomly sampled on 10% of deletions.

WHY B IS CORRECT

HITL checkpoints for irreversible or high-impact actions must occur BEFORE the action, presenting the exact operation and parameters for approval. This prevents the damage rather than merely observing it.

WHY THE OTHER OPTIONS ARE WRONG

- A. Approving after deletion is too late for an irreversible action.
- C. Blanket up-front approval defeats the purpose of per-action oversight.
- D. Sampling leaves 90% of destructive actions ungated.

★ EXAM TIP

Place HITL gates immediately BEFORE irreversible/high-impact actions, showing the concrete operation and parameters. Approvals must precede consequences.

Q25

D1 · Agentic Architecture

An agentic loop occasionally runs indefinitely, calling tools in circles without converging. Which guardrail most directly prevents runaway loops while preserving legitimate multi-step work?

- A Remove all tools so the agent cannot loop.
- B Set a maximum iteration / tool-call budget per task, with a graceful stop-and-escalate when the budget is exceeded. ✓ correct**
- C Lower the model temperature to 0.
- D Increase max_tokens so the agent has room to finish.

WHY B IS CORRECT

A maximum iteration or tool-call budget bounds the loop and provides a controlled exit (stop and escalate) if the agent fails to converge. It stops runaway behavior without forbidding legitimate multi-step tasks.

WHY THE OTHER OPTIONS ARE WRONG

- A. Removing tools disables the agent's core function.
- C. Temperature affects randomness, not loop termination.
- D. More tokens per response doesn't cap the number of loop iterations.

★ EXAM TIP

Bound agentic loops with an iteration/tool-call budget plus a graceful stop-and-escalate. Guardrails should cap runaway behavior, not disable capability.

Q26

D1 · Agentic Architecture

When should you choose 'computer use' (letting Claude control a GUI via screenshots and actions) instead of calling a tool/API?

- A Whenever an API exists, because computer use is always faster.
- B Only when there is no API or programmatic interface and the task genuinely requires interacting with a graphical UI. ✓ correct**
- C For every task, to keep the architecture uniform.
- D Never — computer use is deprecated.

WHY B IS CORRECT

Computer use is a fallback for systems with no API/programmatic access. It is slower, more error-prone, and higher-risk than direct tool/API calls, so it should be reserved for genuinely GUI-only tasks.

WHY THE OTHER OPTIONS ARE WRONG

- A. When an API exists, calling it directly is more reliable and faster than driving a GUI.
- C. Uniformity is not worth the fragility of screenshot-driven control everywhere.
- D. Computer use is a real capability, just niche.

★ EXAM TIP

Prefer APIs/tools over computer use. Reserve computer use for GUI-only systems with no programmatic interface — it's a last resort, not a default.

Q27

D1 · Agentic Architecture

You are choosing between parallel and sequential subagent execution. Which factor most strongly favors SEQUENTIAL execution?

- A The subtasks are fully independent and don't share inputs.
- B You want the fastest possible wall-clock completion.
- C Each subtask depends on the output of the previous one (a pipeline with data dependencies). ✓ correct**
- D You have ample rate limit headroom.

WHY C IS CORRECT

Sequential execution is required when subtasks have data dependencies — each step consumes the previous step's output. Parallelism is only safe/beneficial when subtasks are independent.

WHY THE OTHER OPTIONS ARE WRONG

- A.** Independent subtasks favor parallel execution.
- B.** Wanting speed favors parallel, not sequential.
- D.** Rate-limit headroom enables parallelism, not sequencing.

★ EXAM TIP

Parallelize independent subtasks; sequence dependent ones. Data dependencies are the deciding factor between parallel and sequential orchestration.

D2 · Claude Code

Claude Code Configuration & Workflows · Exam weight 20% · 21 questions

Q28

D2 · Claude Code

A team uses a shared MCP research server. The configuration should be shared across developers, but the API key must NOT be committed to version control. What is the best setup?

- A Hardcode the API key in `.mcp.json`.
- B Pass the API key as a CLI flag on every run.
- C Configure it only in each developer's personal `~/.claude.json`.
- D Configure the server in project-scoped `.mcp.json` using environment-variable expansion (e.g., `${RESEARCH_API_KEY}`) so credentials aren't committed. ✓ correct**

WHY D IS CORRECT

Project-scoped `.mcp.json` shares the server definition with the whole team, while environment-variable expansion keeps the secret out of source control. Each developer supplies the env var locally or via a secrets manager.

WHY THE OTHER OPTIONS ARE WRONG

- A. Hardcoding a secret in a committed file is a security violation.
- B. CLI flags are error-prone and can leak into shell history/process lists.
- C. Personal config keeps the secret local but no longer shares the server configuration with the team.

★ EXAM TIP

Commit configuration, not secrets. Use project `.mcp.json` for shared servers + `${ENV_VAR}` expansion for credentials.

Q29

D2 · Claude Code

A monorepo has `/frontend` (React/TypeScript/Tailwind) and `/backend` (Python/FastAPI/Pydantic). A single root `CLAUDE.md` causes Claude Code to apply TypeScript patterns to Python and vice versa. What is the best fix?

- A Create `/frontend/CLAUDE.md` and `/backend/CLAUDE.md` with each stack's conventions, keeping only shared standards in the root `CLAUDE.md`. ✓ correct**
- B Add 'FOR FRONTEND ONLY' headers throughout one big root `CLAUDE.md`.
- C Delete all `CLAUDE.md` files and rely on the model to infer conventions.
- D Merge frontend and backend into a single language.

WHY A IS CORRECT

Directory-level CLAUDE.md files keep stack-specific conventions close to the code they govern, preventing cross-stack bleed. The root file should hold only shared, organization-wide standards.

WHY THE OTHER OPTIONS ARE WRONG

- B.** Everything still lives in one file, so Claude must sift all conventions and can mis-apply them.
- C.** Deleting guidance loses valuable persistent context.
- D.** Rewriting the architecture to fix instructions is absurd.

★ EXAM TIP

CLAUDE.md is hierarchical. Put shared standards at the root and stack/module-specific rules in per-directory CLAUDE.md files closest to the relevant code.

Q30**D2 · Claude Code**

Which items belong in a PROJECT CLAUDE.md versus a personal ~/.claude/CLAUDE.md: (1) vim keybindings, (2) preferred comment color theme, (3) preference for Hinglish responses, (4) a dentist-appointment reminder?

- A** Put all four in the project root CLAUDE.md.
- B** Put all four in ~/.claude/CLAUDE.md.
- C** Only the coding style guide belongs in the project file; everything else goes elsewhere.

D Items 1-3 are personal preferences and belong in ~/.claude/CLAUDE.md; the dentist appointment belongs in no CLAUDE.md. Project CLAUDE.md holds only coding standards, architecture, and team conventions.
✓ correct

WHY D IS CORRECT

Vim keybindings, comment theme, and response-language preference are personal and belong in the user-level file. A dentist appointment is unrelated to coding and belongs in neither. The shared project file should contain only team-relevant coding guidance.

WHY THE OTHER OPTIONS ARE WRONG

- A.** Personal preferences and a calendar reminder shouldn't pollute shared project config.
- B.** The dentist appointment is irrelevant to Claude's coding behavior even at the user level.
- C.** This ignores that items 1-3 legitimately belong in the personal user file.

★ EXAM TIP

Project CLAUDE.md = team/coding/architecture. ~/.claude/CLAUDE.md = personal coding preferences. Neither = life reminders and calendar events.

Q31

D2 · Claude Code

A dev wants to add to CLAUDE.md: (1) the production DB connection string, (2) a 200-line API spec copied from Confluence, (3) the current sprint's Jira ticket list, and (4) the team coding style guide. Which should actually go in CLAUDE.md?

- A All four.
- B The coding style guide and the API specification.
- C None of them.

D Only the coding style guide — the connection string is a secret, the 200-line spec bloats context, and the sprint list is ephemeral. ✓ correct

WHY D IS CORRECT

CLAUDE.md should hold durable, high-signal guidance. The style guide qualifies. Secrets must never be committed, large copied docs waste context (link/summarize instead), and sprint tickets are transient and quickly stale.

WHY THE OTHER OPTIONS ARE WRONG

- A. Mixes secrets, ephemeral data, and doc bloat with legitimate guidance.
- B. The 200-line spec bloats context; reference it instead of pasting it.
- C. Coding standards are exactly the kind of durable guidance CLAUDE.md is for.

★ EXAM TIP

CLAUDE.md = durable, high-value, non-secret guidance. Keep out secrets, giant pasted docs, and ephemeral sprint data.

Q32

D2 · Claude Code

A 4-step 'prepare a PR' workflow (read changed files, check lint rules, generate missing tests, write a PR description) is run daily by the whole team. What is the best way to standardize it?

- A Document it in CLAUDE.md and hope everyone follows it.
- B Have developers memorize the prompt.
- C Build a shell script that does everything.

D Create a project-level slash command in .claude/commands/prepare-pr.md that encodes the workflow and share it via version control. ✓ correct

WHY D IS CORRECT

Repeated, multi-step, team-wide workflows are the exact use case for a project slash command. It is version-controlled, shared, centrally updatable, and integrates with Claude Code's reasoning steps (reviewing changes, generating tests, writing descriptions).

WHY THE OTHER OPTIONS ARE WRONG

- A. CLAUDE.md is for guidance/conventions, not executable repeatable workflows.
- B. Memorization is inconsistent and error-prone.
- C. A shell script can't perform the Claude-specific reasoning tasks in the workflow.

★ EXAM TIP

If people keep typing the same multi-step prompt, make it a slash command. Project commands live in `.claude/commands/*.md` and are shared via git.

Q33

D2 · Claude Code

A `/generate-tests` slash command is hardcoded to always target `src/utils/helpers.ts`. How do you make it reusable for any file?

- A List every possible file path inside the command.
- B Edit the command file each time before running it.
- C **Use the `$ARGUMENTS` placeholder in the command template so developers can run `'/generate-tests src/auth/login.ts'` and the path is injected dynamically. ✓ correct**
- D Create one slash command per file.

WHY C IS CORRECT

Slash commands support dynamic input via the `$ARGUMENTS` placeholder. Substituting `$ARGUMENTS` with the runtime path makes a single command reusable across the entire codebase.

WHY THE OTHER OPTIONS ARE WRONG

- A. Enumerating paths doesn't scale and needs constant maintenance.
- B. Editing the file each run defeats the purpose of a reusable command.
- D. One command per file quickly becomes unmanageable.

★ EXAM TIP

Whatever part of a command changes each run should become a `$ARGUMENTS` parameter. This is the standard way to make slash commands generic.

Q34

D2 · Claude Code

A developer wrote a `/quick-deploy` command that deploys to their **PERSONAL** staging server. Where should it live?

- A** In the developer's personal `~/.claude/commands/` directory, since it's a personal workflow specific to their environment. ✓ correct
- B In the shared `.claude/commands/` directory with a warning comment.
- C In the project `CLAUDE.md`.
- D In a team wiki page.

WHY A IS CORRECT

The command is tied to one developer's personal environment and could misfire for others. Personal workflows belong in the user-level `~/.claude/commands/` directory, separate from shared team commands.

WHY THE OTHER OPTIONS ARE WRONG

- B.** Warnings are easy to miss; the command shouldn't be shared at all.
- C.** `CLAUDE.md` is for conventions, not executable personal commands.
- D.** A wiki documents but doesn't provide the executable command in the right scope.

★ EXAM TIP

Shared team commands -> project `.claude/commands/`. Personal/environment-specific commands -> `~/.claude/commands/`.

Q35

D2 · Claude Code

A JWT-auth migration touches middleware, route handlers, DB models, and tests (14 files). The chosen approach turns out to be incompatible with the API gateway, but only after 8 files were edited. What should have been done?

- A Split the migration into 14 independent single-file tasks.
- B** Use Plan Mode (Shift+Tab) to generate and review a structured implementation plan — scope and architectural approach — before allowing execution. ✓ correct
- C Write the JWT implementation manually.
- D Review each file as it changes.

WHY B IS CORRECT

Large, cross-cutting, architectural changes should start in Plan Mode. Reviewing the proposed approach and affected files first would have surfaced the API-gateway incompatibility before any files were modified.

WHY THE OTHER OPTIONS ARE WRONG

- A. JWT migration is a coordinated architectural change; splitting into independent tasks risks inconsistency and doesn't validate the approach.
- C. The issue was the architecture choice, not Claude's ability to edit files.
- D. Reviewing per file is too late — the flawed approach was already committed to.

★ EXAM TIP

Use Plan Mode for multi-file/architectural changes. Catch bad approaches during plan review, before execution — that's the whole point of Plan Mode.

Q36

D2 · Claude Code

A developer uses Plan Mode for tiny tasks like fixing a README typo, adding one import, or renaming a CSS class, and finds the overhead annoying. What is the right guidance?

- A Auto-approve all plans to remove friction.
- B Disable Plan Mode entirely.
- C Use direct execution for well-scoped, single-file, low-risk tasks, and reserve Plan Mode for multi-file changes, architectural decisions, and ambiguous tasks. ✓ correct**
- D Auto-execute plans after a 10-second delay.

WHY C IS CORRECT

Plan Mode's value scales with scope, uncertainty, and architectural impact. Small, obvious, single-file edits are better served by direct execution; Plan Mode is for larger or ambiguous work.

WHY THE OTHER OPTIONS ARE WRONG

- A. Auto-approving pays the overhead while removing the benefit.
- B. Plan Mode is valuable for big changes; don't disable it globally.
- D. A timed auto-execute undermines the review step.

★ EXAM TIP

Direct execution for small/low-risk/single-file. Plan Mode for multi-file/architectural/ambiguous. Match the ceremony to the risk.

Q37

D2 · Claude Code

A task is scoped to updating fixtures in `/tests/fixtures/`, but Claude Code also modifies `/src/core/database.py`, introducing a regression. How do you PREVENT out-of-scope modifications?

- A Add 'Only modify files in `/tests/fixtures/`' to the prompt.
- B Scope Claude Code's write permissions to `/tests/fixtures/`, so any attempt to modify files outside it triggers a permission request the developer must approve. ✓ correct**
- C Review the git diff afterward.
- D Use a separate branch and rely on code review.

WHY B IS CORRECT

Prevention comes from permissions, not prompts. Restricting write access to the task directory forces an explicit approval before any out-of-scope file can be touched (least privilege).

WHY THE OTHER OPTIONS ARE WRONG

- A. Prompt instructions are guidance, not enforcement.
- C. Diff review detects the regression after it's introduced.
- D. Branches and reviews catch mistakes later; they don't prevent the out-of-scope edit.

★ EXAM TIP

Enforce task scope with write-permission boundaries, not prompt text. Crossing the boundary should require explicit approval.

Q38

D2 · Claude Code

You want Claude Code to generate DB migration SCRIPTS but must GUARANTEE a human reviews them before any migration runs. What best enforces this?

- A Remove Bash from allowedTools so Claude Code can write migration scripts but cannot execute them; execution becomes a manual, review-required step. ✓ correct**
- B Add nightly backups so migrations can be rolled back.
- C Require running migrations in a separate terminal window.
- D Add an instruction in `CLAUDE.md` to always review migrations first.

WHY A IS CORRECT

Removing the execution capability (Bash) means Claude Code can author scripts but physically cannot run them, forcing a human-executed, review-gated step. Enforcement via permissions beats reliance on process or prompts.

WHY THE OTHER OPTIONS ARE WRONG

- B.** Backups help recover after a mistake; they don't force pre-execution review.
- C.** A separate window is a convention, not an enforced control.
- D.** CLAUDE.md guidance can be ignored; it's not a guarantee.

★ EXAM TIP

To guarantee a human gate on high-risk actions, remove the capability (e.g., Bash) rather than relying on instructions. Permissions enforce; prompts merely guide.

Q39**D2 · Claude Code**

A developer enabled auto-accept mode while building a greenfield service, then switched to a production hotfix on main — and Claude Code auto-approved a command affecting the production database. What is the best fix?

- A** Require two-factor authentication before every tool call.
- B** Permanently disable auto-accept mode.
- C** Add a 5-minute cooldown after switching branches.
- D** **Scope permissions per context: liberal auto-accept for greenfield/experimental branches, and restrictive (explicit approval for writes and bash) for production/main. ✓ correct**

WHY D IS CORRECT

Permissions should be proportional to risk. Relaxed settings are fine for experimental work but production/main should require explicit approval for writes and shell commands. Context-aware permissions preserve velocity while protecting production.

WHY THE OTHER OPTIONS ARE WRONG

- A.** Per-call 2FA is disruptive and doesn't address context-based risk.
- B.** Permanently disabling auto-accept over-penalizes low-risk work.
- C.** Branch switches don't reliably correlate with risk; a cooldown is annoying and unreliable.

★ EXAM TIP

Set permissions by context/risk. Loose on sandboxes, strict on production/main. Never carry a permissive profile into a high-risk context.

Q40

D2 · Claude Code

CLAUDE.md says 'ORM: use the project ORM.' In Plan Mode, Claude Code proposes SQLAlchemy, but the standard is Tortoise ORM. What is the correct two-part fix?

- A Conclude Claude Code doesn't read CLAUDE.md in Plan Mode.
- B Add SQLAlchemy as a second supported ORM.
- C Conclude Plan Mode is unreliable and stop using it.
- D The CLAUDE.md entry is too vague: (1) update it to explicitly name Tortoise ORM as the standard, and (2) review the plan output and redirect it to Tortoise before execution. ✓ correct**

WHY D IS CORRECT

The instruction lacked specificity, so a reasonable generic default (SQLAlchemy) was chosen. Fix the guidance to be explicit AND use Plan Mode's review step to correct the approach before code is generated.

WHY THE OTHER OPTIONS ARE WRONG

- A. Plan Mode does use CLAUDE.md; the entry was just ambiguous.
- B. Adding a second ORM increases inconsistency against the standardized choice.
- C. Plan Mode actually surfaced the issue before execution — it worked as intended.

★ EXAM TIP

Make CLAUDE.md instructions specific and technology-explicit. And remember Plan Mode's benefit: catch wrong assumptions before execution.

Q41

D2 · Claude Code

In a long Claude Code session, config.ts is restructured early, then edited again much later; the second edit partially overwrites the first, breaking compilation. What is the best recovery?

- A Ask Claude Code to fix the compilation error in the same session.
- B Revert the second edit, commit the coherent first edit, and start a fresh session for further config.ts changes — the conflicting edits indicate the session lost coherence. ✓ correct**
- C Manually merge the two edits by hand.
- D Run Claude Code on the broken file in a new session without reverting.

WHY B IS CORRECT

Conflicting edits signal context drift. The safe recovery is to return to a known-good state (revert the bad edit, commit the good one) and continue in a fresh session that reasons from the clean committed state.

WHY THE OTHER OPTIONS ARE WRONG

- A. Continuing in the same drifted session risks more inconsistencies.
- C. Manual merge treats the symptom, not the lost coherence.
- D. A new session helps, but starting from a corrupted hybrid state forces reasoning over code that should never have existed.

★ EXAM TIP

On context drift/conflicting edits: revert to known-good, commit, then start a fresh session. Use git commits as checkpoints; don't fight a drifted session.

Q42

D2 · Claude Code

Across a multi-day project, later Claude Code sessions drift from conventions set on day one (camelCase becomes snake_case, error handling diverges, import order changes). What design most reliably prevents this drift?

- A Use one continuous 3-day session.
- B Have a developer review every single line manually.
- C Copy Session 1's output into every later session's prompt.
- D Encode naming conventions, error-handling patterns, and import ordering in CLAUDE.md so they load automatically every session as a consistent anchor. ✓ correct**

WHY D IS CORRECT

Persistent, explicit conventions in CLAUDE.md load on every session, giving a stable anchor regardless of how many sessions or developers are involved — the intended mechanism for cross-session consistency.

WHY THE OTHER OPTIONS ARE WRONG

- A. Very long sessions increase context drift, not reduce it.
- B. Line-by-line review is costly and doesn't prevent drift from occurring.
- C. Manual copy-paste is error-prone; CLAUDE.md already provides this automatically.

★ EXAM TIP

Cross-session consistency = encode conventions in CLAUDE.md. It loads automatically each session and anchors naming, error handling, and imports.

Q43

D2 · Claude Code

A style inconsistency introduced in Session 1 was committed unreviewed; Sessions 2 and 3 then treated it as the norm and spread it. What practice best prevents this propagation?

- A Never commit Claude Code output directly; rewrite everything by hand.
- B Run an automated linter after every commit.
- C Review the git diff at the end of each session before committing, using the session boundary as a checkpoint to catch and correct drift before it propagates. ✓ correct**
- D Use a separate branch per session.

WHY C IS CORRECT

Because Claude Code uses existing committed code as context, unreviewed inconsistencies become the new baseline. Reviewing the diff at each session boundary and committing only clean code stops bad patterns from spreading.

WHY THE OTHER OPTIONS ARE WRONG

- A. Rewriting everything discards most of Claude Code's value; the issue is missing review.
- B. Linters catch formatting but miss architectural/naming/error-handling drift.
- D. Branching doesn't prevent inconsistent patterns from being merged later.

★ EXAM TIP

Treat each session boundary as a quality gate: review the diff, fix issues, commit clean. Clean commits become clean future context.

Q44

D2 · Claude Code

The same project context (error-handling conventions, preferred logging library, test-naming patterns, migration workflow) is needed identically in every session by every developer. What is the best way to provide it?

- A Document it in the project's CLAUDE.md, which loads automatically at the start of every session. ✓ correct**
- B Create a shared prompt template developers paste in each session.
- C Train developers to type the conventions from memory.
- D Build a custom plugin to inject the context.

WHY A IS CORRECT

Stable, project-wide conventions belong in CLAUDE.md, which Claude Code loads automatically each session — eliminating repetitive re-explanation for every developer.

WHY THE OTHER OPTIONS ARE WRONG

- B.** A template still relies on remembering to paste it each time.
- C.** Typing from memory is inconsistent and error-prone.
- D.** A plugin is unnecessary complexity when CLAUDE.md already handles this.

★ EXAM TIP

Identical-every-session context -> CLAUDE.md. Don't re-paste; let the file load automatically.

Q45**D2 · Claude Code**

A monorepo has /api/v1 (REST/JSON), /api/v2 (GraphQL), and /api/internal (gRPC/protobuf). A single root CLAUDE.md causes REST conventions to bleed into GraphQL code. What is the best fix?

A

Create /api/v1/CLAUDE.md, /api/v2/CLAUDE.md, and /api/internal/CLAUDE.md, each with only that module's conventions, keeping shared standards in the root. ✓ correct

B

Add 'FOR /api/v1 ONLY' headers within one root file.

C

Create a single /api/CLAUDE.md combining all three convention sets.

D

Replace CLAUDE.md with scattered code comments.

WHY A IS CORRECT

Directory-scoped CLAUDE.md files keep each API style's conventions next to its code, preventing convention bleed. The root retains only cross-module shared standards.

WHY THE OTHER OPTIONS ARE WRONG

- B.** One file still forces Claude to sift all conventions and risks mis-application.
- C.** Combining all three in one /api file doesn't solve the bleed problem.
- D.** Comments are scattered and not a substitute for structured guidance.

★ EXAM TIP

Put instructions as close as possible to the code they govern. Per-module CLAUDE.md > one giant conditional file.

Q46

D2 · Claude Code

A request to 'add input validation to the signup form' turns out to require changes across the form component, API endpoint, DB schema, error constants, and tests (6 files) mid-execution. What is the best response?

- A** Stop execution, switch to Plan Mode, and have Claude Code produce a complete plan for the multi-file change before resuming. ✓ correct
- B Let Claude finish all 6 files and review afterward.
- C Revert and force the change into only the form component.
- D Continue because progress has already been made.

WHY A IS CORRECT

The task outgrew its original scope into a cross-cutting change. The moment it becomes multi-file and interconnected, switch to Plan Mode to review scope and approach before continuing.

WHY THE OTHER OPTIONS ARE WRONG

- B.** Finishing first risks propagating wrong assumptions across files.
- C.** Validation legitimately spans layers; forcing it into one file leaves the system inconsistent.
- D.** Sunk-cost reasoning; scope change warrants re-planning.

★ EXAM TIP

If a 'small' task expands into multi-file/architectural work mid-flight, stop and switch to Plan Mode. Re-plan when scope changes.

Q47

D2 · Claude Code

What is the fundamental difference between a bundled command like `/batch` and a custom slash command like `/project:security-review`?

- A Custom commands are faster because they bypass Claude's safety checks.
- B** Bundled commands are maintained by Anthropic, ship with every install, and can access internal capabilities like spawning agents in isolated worktrees; custom commands are project-specific markdown prompts that leverage codebase context but cannot access those internal APIs. ✓ correct
- C Custom commands cost more API tokens per call.
- D There is no difference — both are plain markdown files.

WHY B IS CORRECT

Bundled commands are Anthropic-maintained, shipped with Claude Code, and can use privileged internal capabilities (e.g., spawning agents in isolated worktrees). Custom commands are user/team markdown prompts that use codebase context and standard capabilities but not internal APIs.

WHY THE OTHER OPTIONS ARE WRONG

- A. Custom commands do not bypass safety checks or permissions.
- C. Token cost depends on the task, not whether a command is bundled or custom.
- D. Custom commands are markdown, but bundled commands have built-in capabilities beyond plain markdown.

★ EXAM TIP

Bundled = Anthropic-maintained with internal capabilities. Custom = your markdown prompts leveraging codebase context, no internal APIs.

Q48

D2 · Claude Code

You have a reusable security-review rubric you want Claude to apply automatically whenever it reviews code, without pasting it each time and without consuming context until needed. Which Claude Code primitive fits best?

- A An MCP server.
- B A subagent.
- C **A Skill (SKILL.md) whose description sits in context cheaply and whose full body loads only when the situation matches. ✓ correct**
- D A larger CLAUDE.md.

WHY C IS CORRECT

Skills change HOW Claude does a task. A SKILL.md's ~100-token description stays in context, but its full rubric loads only when relevant — perfect for a reusable review procedure applied on demand.

WHY THE OTHER OPTIONS ARE WRONG

- A. MCP servers add access to external systems; a rubric is instructions, not a connection.
- B. Subagents protect context for noisy tasks; you don't need an isolated context just to apply a rubric.
- D. A bigger CLAUDE.md loads its content on every session regardless of relevance, wasting context.

★ EXAM TIP

Skills change behavior, subagents protect context, MCP adds capability. A reusable on-demand procedure = a Skill.

D3 · Prompt Engineering

Prompt Engineering & Structured Output · Exam weight 20% · 19 questions

Q49

D3 · Prompt Engineering

Claude sometimes returns a fabricated PO number like 'PO-12345' when an invoice has none. What is the best schema-level fix?

- A Default po_number to 'UNKNOWN'.
- B Remove the po_number field entirely.
- C Tell Claude to 'be honest' in the system prompt.
- D Add a per-property description ('exactly as it appears; return null if absent; do not infer or generate') AND make the field nullable (type: ['string','null']) so absence is a first-class value. ✓ correct**

WHY D IS CORRECT

Making 'absent' a valid, schema-supported outcome (nullable + explicit 'return null if not present, do not infer') removes the pressure to invent a value. Schema constraints are stronger than prompt pleas.

WHY THE OTHER OPTIONS ARE WRONG

- A. 'UNKNOWN' is just another invented value and loses the null/absent distinction.
- B. Removing the field loses valid PO numbers when they exist.
- C. A generic 'be honest' instruction is weaker than an explicit schema constraint.

★ EXAM TIP

For optional fields, make null a first-class value (type: ['string','null']) and instruct 'return null if absent; do not infer.' Encode extraction rules in the schema, not just the prompt.

Q50

D3 · Prompt Engineering

A document_type field should only be invoice, credit_note, purchase_order, or delivery_note, but Claude returns variants like 'Tax Invoice,' 'INV,' and 'credit memo.' What is the cleanest fix?

- A Validate with a regex pattern.
- B Use numeric codes (1,2,3,4).
- C Keep free text and normalize downstream.
- D Add an enum with the four allowed values plus a per-property description defining each, so Claude maps variants to canonical values. ✓ correct**

WHY D IS CORRECT

An enum constrains output to a closed, canonical set, and per-value descriptions guide Claude to map 'Tax Invoice' -> invoice, etc. This enforces correctness at extraction time.

WHY THE OTHER OPTIONS ARE WRONG

- A. Regex validates format but doesn't give a canonical vocabulary.
- B. Numeric codes hurt readability and require downstream mapping.
- C. Free text plus downstream normalization lets inconsistency persist.

★ EXAM TIP

Classification into a fixed set = enum + clear per-value descriptions. Let the schema constrain the model to canonical values.

Q51

D3 · Prompt Engineering

A service_period is extracted as free text like 'for services rendered from 1 Jan 2026 to 31 Mar 2026.' Downstream systems can't filter or aggregate it. What is the best schema fix?

- A Use two free-text fields service_start and service_end.
- B Drop the field.
- C Replace it with a structured object {start_date, end_date}, both constrained to ISO 8601, enabling filtering, aggregation, and overlap detection. ✓ correct**
- D Keep the free-text field and let consumers parse it.

WHY C IS CORRECT

A structured object with ISO 8601 start_date/end_date turns natural-language ranges into queryable data supporting filtering, aggregation, and overlap detection.

WHY THE OTHER OPTIONS ARE WRONG

- A. Two free-text fields still leave formatting ambiguity and validation problems.
- B. The service period is valuable business data; don't drop it.
- D. Free text pushes parsing complexity onto every consumer.

★ EXAM TIP

Structure semantic data early. Use ISO 8601 for dates and explicit start/end fields instead of natural-language ranges.

Q52

D3 · Prompt Engineering

Invoices arrive from 14 countries in different currencies. The extracted total is a bare number, so downstream consumers can't tell if 1500 is GBP, EUR, or INR. What is the best schema fix?

- A Always coerce totals to USD in the extractor.
- B Assume the currency from the vendor's country.
- C Add a sibling currency field constrained to ISO 4217 codes via an enum, and require both total and currency together in the schema. ✓ correct**
- D Use a string like 'GBP 1500' for the total.

WHY C IS CORRECT

Storing amount and currency as separate structured fields — currency as an ISO 4217 enum, both required together — makes monetary values unambiguous and usable downstream.

WHY THE OTHER OPTIONS ARE WRONG

- A. Coercing to USD destroys the original value and requires FX assumptions.
- B. Vendors often bill in a currency different from their country; never infer currency when it can be extracted.
- D. Embedding currency and amount in one string forces repeated parsing downstream.

★ EXAM TIP

Money = separate amount + ISO 4217 currency code, required together. Don't infer currency and don't cram it into a string.

Q53

D3 · Prompt Engineering

A schema has subtotal, tax, and total where $\text{total} = \text{subtotal} + \text{tax}$. Should Claude extract all three, or extract two and compute the third?

- A Extract subtotal and tax, then compute total yourself.
- B Extract all three and ignore any inconsistency.
- C Extract total only.
- D Extract all three as they appear (they are document facts, not derived values), then validate the relationship downstream and flag mismatches, which often reveal rounding, hidden discounts, or extraction errors. ✓ correct**

WHY D IS CORRECT

Extraction should capture facts as printed. Keeping all three lets a downstream validation step detect discrepancies (rounding, discounts, errors) that computing the total would silently hide.

WHY THE OTHER OPTIONS ARE WRONG

- A. Computing total discards the printed value and its evidentiary/validation value.
- B. Including all three but ignoring inconsistency wastes a valuable quality signal.
- C. Total-only loses reconciliation ability and useful data.

★ EXAM TIP

Extract document facts verbatim; validate relationships separately. Mismatches are signals, not reasons to overwrite extracted data.

Q54

D3 · Prompt Engineering

Extracted 'notes' frequently include 'CONFIDENTIAL' watermarks, page numbers, and repeated headers. What is the cleanest fix?

- A Ignore the noise; consumers can handle it.
- B Increase the model's temperature.
- C Filter the noise downstream with regex.

D Either pre-process the document to strip structural noise (page numbers, watermarks, repeated headers/footers) before extraction, or scope the field description so notes contain only buyer/seller body communication, not boilerplate. ✓ correct

WHY D IS CORRECT

This is an input-quality problem. Remove structural artifacts before extraction and/or tightly scope the field definition so the model knows exactly what belongs in notes. Clean inputs beat post-hoc cleanup.

WHY THE OTHER OPTIONS ARE WRONG

- A. Ignoring noise pushes cleanup to every consumer and lowers quality.
- B. Higher temperature increases variability and usually worsens extraction consistency.
- C. Downstream regex is brittle and duplicated across consumers; prefer upstream cleaning/scoping.

★ EXAM TIP

Clean or scope inputs before extraction. Strip page numbers/watermarks/headers, or precisely define the field so boilerplate is excluded.

Q55

D3 · Prompt Engineering

You need Claude to reliably reason through a complex multi-step calculation before giving a final structured answer. Which prompting technique best improves accuracy?

- A Ask only for the final JSON to save tokens.
- B Prompt Claude to think step by step (chain-of-thought), then provide the final answer — optionally separating reasoning in <thinking> tags from the final structured output. ✓ correct**
- C Raise the temperature to encourage creativity.
- D Repeat the question three times in the prompt.

WHY B IS CORRECT

Chain-of-thought (explicit step-by-step reasoning) improves accuracy on complex tasks. Separating reasoning (e.g., in <thinking> tags) from the final structured output keeps the machine-readable result clean.

WHY THE OTHER OPTIONS ARE WRONG

- A. Forcing an immediate answer suppresses reasoning and hurts accuracy on multi-step problems.
- C. Higher temperature adds randomness, not reliability.
- D. Repetition doesn't add reasoning capacity.

★ EXAM TIP

For complex reasoning, elicit step-by-step thinking, then a clean final answer. Use tags/sections to separate reasoning from structured output.

Q56

D3 · Prompt Engineering

Which technique most reliably forces Claude's structured output to begin in a specific format (e.g., start a JSON object immediately with no preamble)?

- A Politely ask 'please respond only in JSON.'
- B Prefill the assistant turn with the opening token(s) of the desired format (e.g., '{') so Claude continues from there. ✓ correct**
- C Set max_tokens very low.
- D Add many exclamation points to the instruction.

WHY B IS CORRECT

Assistant response prefilling — starting the assistant turn with the first characters of the desired output (like '{') — strongly constrains Claude to continue in that format and skip preambles.

WHY THE OTHER OPTIONS ARE WRONG

- A. A polite request helps but is less reliable than prefilling.
- C. Low max_tokens truncates output; it doesn't shape the format.
- D. Punctuation doesn't constrain output format.

★ EXAM TIP

Prefill the assistant turn (e.g., with '{' or '<result>') to force a format and eliminate preambles. It's the most reliable formatting lever.

Q57

D3 · Prompt Engineering

You want to steer Claude with 3 worked examples of the exact input->output mapping you expect. What is this technique called and when is it most useful?

- A Zero-shot prompting; best when the task is novel and undefined.
- B Few-shot (multishot) prompting; best when you can demonstrate the desired format and edge-case handling with examples. ✓ correct**
- C Temperature tuning; best for deterministic tasks.
- D Prompt caching; best for reducing latency.

WHY B IS CORRECT

Providing several input/output examples is few-shot (multishot) prompting. It's most useful when concrete examples clarify the exact format, style, and edge-case handling you expect.

WHY THE OTHER OPTIONS ARE WRONG

- A. Zero-shot means no examples; the scenario explicitly provides examples.
- C. Temperature controls randomness, not demonstration.
- D. Prompt caching optimizes cost/latency, not example-based steering.

★ EXAM TIP

Examples = few-shot/multishot prompting. Use 2-5 diverse examples that cover the format and tricky edge cases to lock in behavior.

Q58

D3 · Prompt Engineering

A long, complex prompt mixes instructions, a big reference document, and the user's question, and Claude sometimes confuses them. What is the recommended structuring technique?

- A Delete the reference document to reduce confusion.
- B Use XML tags to clearly delimit sections (e.g., <instructions>, <document>, <question>) so Claude can distinguish them. ✓ correct**
- C Put everything on a single line.
- D Lower max_tokens.

WHY B IS CORRECT

XML tags are a recommended way to structure complex prompts, clearly separating instructions, reference material, and the question so Claude can parse roles unambiguously.

WHY THE OTHER OPTIONS ARE WRONG

- A. The document is needed; the issue is structure, not its presence.
- C. One line worsens readability and separation.
- D. Token limits don't clarify structure.

★ EXAM TIP

Use XML tags (<instructions>, <context>, <document>, <question>) to delimit prompt sections. Clear boundaries reduce confusion in long prompts.

Q59

D3 · Prompt Engineering

Which practice most improves the reliability of a system prompt that defines an agent's role and rules?

- A Keep it vague so the model has flexibility.
- B Give a clear role, explicit rules/constraints, and concrete guidance on tone and edge cases, placing durable instructions in the system prompt. ✓ correct**
- C Put all instructions in the final user turn instead.
- D Rely on emojis to convey tone.

WHY B IS CORRECT

A strong system prompt establishes a clear role, explicit constraints, and concrete guidance for tone and edge cases. Durable, role-defining instructions belong in the system prompt where they consistently shape behavior.

WHY THE OTHER OPTIONS ARE WRONG

- A. Vagueness reduces reliability and consistency.
- C. Putting durable rules only in a user turn makes them easy to lose across turns.
- D. Emojis are not a substitute for explicit tone guidance.

★ EXAM TIP

System prompt = role + explicit rules + tone + edge-case guidance. Keep durable behavior there; keep per-request details in the user turn.

Q60

D3 · Prompt Engineering

You want stable, machine-parseable JSON from Claude across many documents. Beyond a good prompt, what most improves structural reliability?

- A Ask for Markdown instead of JSON.
- B Define an explicit output schema (required fields, types, enums, nullability) and validate every response against it, retrying or flagging on failure. ✓ correct**
- C Increase temperature for variety.
- D Let the model choose whatever structure it likes each time.

WHY B IS CORRECT

Defining an explicit schema (types, required fields, enums, nullability) and validating each response — with retries or flagging on failure — is the reliable way to get consistent, machine-parseable JSON at scale.

WHY THE OTHER OPTIONS ARE WRONG

- A. Markdown is less machine-parseable than schema-validated JSON.
- C. Higher temperature reduces consistency.
- D. Free-form structure defeats machine parsing.

★ EXAM TIP

Pair a schema with runtime validation. Constrain (schema) + verify (validate) + recover (retry/flag) = reliable structured output.

Q61

D3 · Prompt Engineering

For a Claude request that must return either a valid answer or an explicit 'cannot determine' when evidence is missing, what prompt design best prevents confident hallucination?

- A Tell Claude it must always provide a definitive answer.
- B Explicitly permit and instruct an 'unknown/cannot determine' output when evidence is insufficient, and describe what counts as insufficient. ✓ correct**
- C Raise temperature so answers vary.
- D Ask the question twice.

WHY B IS CORRECT

Giving the model an explicit, legitimate 'I don't know / cannot determine' path — with criteria for when to use it — reduces pressure to fabricate and improves calibration.

WHY THE OTHER OPTIONS ARE WRONG

- A. Demanding a definitive answer encourages hallucination when evidence is missing.
- C. Temperature adds randomness, not honesty.
- D. Repetition doesn't create an out for insufficient evidence.

★ EXAM TIP

Always give the model a sanctioned 'unknown' escape hatch with clear criteria. Forcing certainty invites hallucination.

Q62

D3 · Prompt Engineering

When is Claude's extended thinking most appropriate to enable?

- A For trivial lookups where latency and cost matter most.
- B For hard, multi-step reasoning problems (complex math, planning, deep analysis) where more deliberate reasoning improves correctness and the extra latency/cost is justified. ✓ correct**
- C For every request, to be safe.
- D Only when the output must be JSON.

WHY B IS CORRECT

Extended thinking helps on genuinely hard, multi-step problems where deeper deliberation improves accuracy and the added latency/cost is worthwhile.

WHY THE OTHER OPTIONS ARE WRONG

- A. For trivial lookups the added latency/cost isn't justified.
- C. Always-on extended thinking wastes cost/latency on easy tasks.
- D. Output format (JSON) doesn't determine whether extended thinking helps.

★ EXAM TIP

Turn on extended thinking for hard reasoning/planning tasks; skip it for simple lookups. Match reasoning depth to problem difficulty.

Q63

D3 · Prompt Engineering

A schema field 'invoice_number' occasionally captures a random reference or PO number from elsewhere on the page. What prompt/schema change most improves precision?

- A Add temperature.
- B Add a precise per-property description explaining exactly where the invoice number appears and what it is NOT (e.g., not a PO or reference number), plus a format pattern if known. ✓ correct**
- C Rename the field to 'number'.
- D Remove the field.

WHY B IS CORRECT

A precise field description — defining what the value is, where it appears, and what it is not — plus an optional format pattern sharply improves extraction precision and reduces cross-field confusion.

WHY THE OTHER OPTIONS ARE WRONG

- A. Temperature doesn't improve targeting precision.
- C. A vaguer name makes confusion worse.
- D. Removing the field loses needed data.

★ EXAM TIP

Write field descriptions that say what the value IS, where it appears, and what it is NOT. Disambiguation lives in per-property descriptions.

Q64

D3 · Prompt Engineering

You must classify support tickets into fixed categories and also return a short rationale. What output design keeps results reliable AND explainable?

- A Return a single free-text paragraph blending category and reasoning.
- B Return a structured object with an enum 'category' field and a separate 'rationale' string field. ✓ correct**
- C Return only the rationale and let downstream infer the category.
- D Return the category as a number with no rationale.

WHY B IS CORRECT

Separating a constrained enum 'category' from a free-text 'rationale' gives machine-reliable classification plus human-readable explanation, without mixing the two.

WHY THE OTHER OPTIONS ARE WRONG

- A. A blended paragraph is hard to parse reliably.
- C. Inferring the category downstream reintroduces ambiguity.
- D. A bare number drops the required rationale and hurts readability.

★ EXAM TIP

Separate the machine field (enum) from the human field (rationale). Constrain what must be parsed; free-text only what must be read.

Q65

D3 · Prompt Engineering

Which is the best way to reduce a prompt's sensitivity to injection from untrusted document text you must include?

- A Trust the document and merge it directly into your instructions.
- B Clearly separate untrusted content in a delimited section (e.g., XML tags) and instruct Claude to treat it as data to analyze, not as instructions to follow. ✓ correct**
- C Raise temperature to dilute the injection.
- D Shorten your system prompt.

WHY B IS CORRECT

Delimiting untrusted content and explicitly instructing the model to treat it as data (not commands) mitigates prompt injection by preserving the boundary between your instructions and external text.

WHY THE OTHER OPTIONS ARE WRONG

- A. Merging untrusted text into instructions invites injection.
- C. Temperature has no effect on injection resistance.
- D. A shorter system prompt doesn't isolate untrusted content.

★ EXAM TIP

Wrap untrusted input in tags and tell Claude it's data to analyze, not instructions to obey. Maintain the instruction/data boundary.

Q66

D3 · Prompt Engineering

For an extraction schema, why is it better to mark truly optional fields as nullable rather than always required?

- A. Nullable fields are faster to parse.
- B. Requiring every field pressures the model to fabricate values when data is genuinely absent; nullable fields let it correctly represent 'not present.' ✓ correct**
- C. Nullable fields reduce token cost.
- D. Required fields are deprecated.

WHY B IS CORRECT

Marking optional fields nullable lets the model honestly represent missing data instead of inventing values to satisfy a 'required' constraint, improving accuracy.

WHY THE OTHER OPTIONS ARE WRONG

- A. Parsing speed is not the reason.
- C. Token cost isn't the driver.
- D. Required fields are still valid and appropriate for truly mandatory data.

★ EXAM TIP

Required for genuinely mandatory data; nullable for optional. Over-requiring fields is a common cause of hallucinated values.

Q67

D3 · Prompt Engineering

You want consistent tone and policy across thousands of customer replies. Where should the tone/policy rules primarily live?

- A Repeated in every user message.
- B In a well-crafted system prompt that defines tone, policy, and constraints once for the whole conversation.
✓ correct**
- C In the tool schemas.
- D In the model's temperature setting.

WHY B IS CORRECT

Durable tone and policy belong in the system prompt, which consistently governs behavior across the conversation without per-message repetition.

WHY THE OTHER OPTIONS ARE WRONG

- A. Repeating rules in every user message is wasteful and error-prone.
- C. Tool schemas define tools, not conversational tone/policy.
- D. Temperature controls randomness, not tone/policy.

★ EXAM TIP

Durable behavior (tone, policy, role) -> system prompt. Per-request specifics -> user turn. Don't repeat durable rules every message.

D4 · Tool Design & MCP

Tool Design & MCP Integration · Exam weight 18% · 18 questions

Q68**D4 · Tool Design & MCP**

A `web_search` tool returns 200+ fields per result, but downstream agents only need snippet, URL, and publication date. The verbose payload bloats context. What is the best fix?

- A** Convert all raw fields into a single base64 string.
- B** Have each downstream agent re-call `web_search` as needed.
- C** Have the tool return a curated subset (snippet, URL, publication date) instead of the full raw response, so verbose fields don't accumulate in downstream context. ✓ correct
- D** Increase the context window on every downstream agent.

WHY C IS CORRECT

Tools should return the minimum information necessary. Curating the payload at the tool boundary reduces tokens, latency, cost, and noise for every downstream consumer.

WHY THE OTHER OPTIONS ARE WRONG

- A.** Base64 doesn't reduce tokens (it increases size) and still needs decoding.
- B.** Re-calling repeats work and cost.
- D.** Bigger windows treat the symptom; the fix is not to emit unnecessary tokens.

★ EXAM TIP

Filter at the tool boundary. Return only fields consumers need — minimizing context is a core tool-design principle.

Q69**D4 · Tool Design & MCP**

A team runs MCP server credentials inside a GitLab CI/CD pipeline. Where should the credentials be stored?

- A** In a public `.env` file committed to the repo.
- B** Hardcoded in `.gitlab-ci.yml`.
- C** As masked/protected CI/CD variables in GitLab project or group settings, injected at runtime. ✓ correct
- D** In the MCP server's source-code repository.

WHY C IS CORRECT

Secrets should live in the CI/CD platform's secret store. GitLab masked/protected variables keep values out of source control and logs and inject them at runtime.

WHY THE OTHER OPTIONS ARE WRONG

- A. A committed .env exposes secrets to anyone with repo access.
- B. Hardcoding puts secrets in source history.
- D. Storing secrets in code repos violates separation of secrets and code.

★ EXAM TIP

Configuration can be version-controlled; secrets go in the CI/CD secret store (masked/protected variables), injected at runtime.

Q70

D4 · Tool Design & MCP

For running Claude Code in CI WITHOUT directly using an Anthropic API key in an AWS-based organization, what is the recommended integration?

- A Use a --provider aws flag.
- B Integrate with Amazon Bedrock using AWS credentials/IAM. ✓ correct**
- C Use a CLAUDE_AWS_KEY environment variable.
- D AWS integration is not supported.

WHY B IS CORRECT

Claude models run on Amazon Bedrock, letting organizations authenticate with AWS IAM credentials/roles instead of an Anthropic API key — ideal for AWS-based CI with existing governance.

WHY THE OTHER OPTIONS ARE WRONG

- A. There is no simple --provider aws switch that enables Bedrock auth.
- C. There is no standard CLAUDE_AWS_KEY mechanism; Bedrock uses AWS credentials/roles.
- D. AWS integration via Bedrock is a major supported path.

★ EXAM TIP

Platform-to-auth mapping: Anthropic API -> API key; Amazon Bedrock -> AWS/IAM; Google Vertex AI -> GCP credentials. Enterprise/AWS + no API key -> Bedrock.

Q71

D4 · Tool Design & MCP

In the Model Context Protocol, which three server primitives can an MCP server expose to a client?

- A Functions, variables, and classes.
- B Tools, resources, and prompts. ✓ correct**
- C Agents, workers, and coordinators.
- D Endpoints, webhooks, and queues.

WHY B IS CORRECT

MCP defines three server primitives: tools (callable actions), resources (readable data/context), and prompts (reusable prompt templates). These connect Claude to external systems in a standard way.

WHY THE OTHER OPTIONS ARE WRONG

- A. These are generic programming constructs, not MCP primitives.
- C. These are agent roles, not MCP server primitives.
- D. These are integration patterns, not MCP primitives.

★ EXAM TIP

Memorize MCP's three primitives: Tools (do things), Resources (read things), Prompts (reusable templates). Host <-> Client <-> Server is the architecture.

Q72

D4 · Tool Design & MCP

Under the current MCP spec, which transport should you default to for a REMOTE MCP server versus a LOCAL one?

- A SSE for remote; WebSocket for local.
- B Streamable HTTP for remote; stdio for local. ✓ correct**
- C stdio for remote; Streamable HTTP for local.
- D gRPC for remote; TCP for local.

WHY B IS CORRECT

Current MCP guidance recommends Streamable HTTP as the transport for remote servers and stdio for local servers. (SSE is deprecated for new servers.)

WHY THE OTHER OPTIONS ARE WRONG

- A. SSE is deprecated for new remote servers; WebSocket isn't the local default.
- C. This reverses the correct mapping.
- D. gRPC/TCP are not the MCP transport recommendations.

★ EXAM TIP

Remote MCP -> Streamable HTTP. Local MCP -> stdio. SSE is deprecated for new servers.

Q73**D4 · Tool Design & MCP****What most improves an LLM's ability to call a tool correctly?**

- A A short, ambiguous name with no description.
- B A clear tool name, a precise natural-language description of when/how to use it, and a well-typed input schema with per-parameter descriptions. ✓ correct**
- C As many optional parameters as possible for flexibility.
- D Returning the largest possible payload.

WHY B IS CORRECT

Models select and populate tools based on names, descriptions, and typed input schemas. Clear naming, a precise 'when to use' description, and well-documented, typed parameters dramatically improve correct tool use.

WHY THE OTHER OPTIONS ARE WRONG

- A. Ambiguity leads to wrong or missed tool calls.
- C. Too many optional params increase confusion and error rates.
- D. Large payloads bloat context and don't help tool selection.

★ EXAM TIP

Design tools like good API docs: clear name, explicit 'use when...' description, typed params with descriptions. The schema teaches the model how to call correctly.

Q74

D4 · Tool Design & MCP

A tool returns raw backend errors like a stack trace on failure, confusing the agent. What is the best tool-design practice?

- A Suppress all errors and return an empty result.
- B Return structured, actionable error responses (error type/category plus a clear message and suggested next step) so the agent can choose the right recovery. ✓ correct**
- C Crash the agent so a human notices.
- D Return the full stack trace every time.

WHY B IS CORRECT

Tools should return structured, actionable errors (category + message + suggested next step). This lets the agent map the error type to the right recovery (retry, escalate, fix input) instead of guessing.

WHY THE OTHER OPTIONS ARE WRONG

- A. Swallowing errors hides real failures and causes silent wrong behavior.
- C. Crashing is not graceful and loses context.
- D. Raw stack traces are noisy and hard for the agent to act on.

★ EXAM TIP

Design tool errors for the caller: category + human-readable message + recommended next action. Structured errors enable structured recovery.

Q75

D4 · Tool Design & MCP

A Message Batches API job finishes, but 12% of results are missing for a set of files. What is the most likely cause and correct handling?

- A The API key expired mid-batch.
- B The batch exceeded 24 hours and split into two nights.
- C Some individual requests failed; check each result's is_error field and implement retry logic to resubmit only the failed items. ✓ correct**
- D Claude decided to skip low-risk files.

WHY C IS CORRECT

Individual requests in a batch can fail even when the batch completes. Inspect each result's is_error flag, identify failures, and retry just those items. Batch completion does not guarantee every request succeeded.

WHY THE OTHER OPTIONS ARE WRONG

- A. An invalid key would typically fail submission, not lose a partial 12%.
- B. The API doesn't 'split into two nights'; that doesn't explain a 12% loss.
- D. Claude doesn't autonomously skip submitted requests; failures surface via errors, not silent omission.

★ EXAM TIP

Never assume batch completion = all succeeded. Check per-result `is_error` and retry failed items. Design for partial failure.

Q76

D4 · Tool Design & MCP

A document-analysis agent has a generic `fetch_url`(any URL) tool and starts using it to run ad-hoc web searches — a job that belongs to the web-search agent. What is the best fix?

- A Add known search-engine domains to a blocklist.
- B Replace `fetch_url` with a `load_document` tool that validates the URL points to a document format, enforcing the agent's role at the tool boundary. ✓ correct**
- C Route all URL loading through the coordinator.
- D Add a prompt instruction telling it not to search the web.

WHY B IS CORRECT

Give the agent a purpose-built tool. A `load_document` tool that validates document sources prevents role drift far better than a generic `fetch_url`, without creating a bottleneck.

WHY THE OTHER OPTIONS ARE WRONG

- A. Blocklists are reactive and incomplete; non-search sites can still be misused.
- C. Routing everything through the coordinator removes useful autonomy and creates a bottleneck.
- D. Prompt instructions are weaker than a constrained tool.

★ EXAM TIP

Enforce an agent's role via purpose-built tools with input validation, not generic tools plus prompt warnings.

Q77

D4 · Tool Design & MCP

You want to invoke Claude Code in a CI pipeline to review a PR diff for security issues and post a machine-parseable summary. How should it be invoked?

- A Open an interactive Claude Code terminal session in the runner and type the prompt manually.
- B Install the VS Code extension on the runner and trigger it via the GUI.
- C Use headless mode (`claude -p "Review this diff for security vulnerabilities: $(git diff main)"`) with `--output-format json` for machine-parseable results. ✓ correct**
- D Commit the diff to a file and let CLAUDE.md trigger a review on the next push.

WHY C IS CORRECT

CI is non-interactive. Headless mode (`claude -p ...`) with the diff as context and `--output-format json` produces structured results the pipeline can parse and post as a comment.

WHY THE OTHER OPTIONS ARE WRONG

- A. Interactive sessions can't run in automated CI.
- B. The VS Code extension is for developer workstations, not headless runners.
- D. CLAUDE.md provides guidance; it doesn't trigger reviews or pipeline actions.

★ EXAM TIP

Automation = headless mode: `claude -p` with piped context and `--output-format json`. Interactive/IDE modes are for humans, not CI.

Q78

D4 · Tool Design & MCP

A Claude Code CI review step calls a GitHub MCP server, but the MCP call fails silently, so reviews sometimes proceed on incomplete assumptions. What is the most robust fix?

- A Add 'sleep 30' before the MCP call.
- B Disable MCP entirely in CI.
- C Wrap MCP tool calls in error handling: on failure, fall back to analyzing the local diff only and log a clear warning — never let an MCP failure silently corrupt review output. ✓ correct**
- D Retry the entire CI job from scratch on any failure.

WHY C IS CORRECT

External tool calls must fail safely and visibly. Wrap MCP calls, detect failures, log a warning, and degrade gracefully (local-diff-only review) so a silent failure can't misrepresent results.

WHY THE OTHER OPTIONS ARE WRONG

- A. A fixed sleep doesn't handle failures or make them visible.
- B. Disabling MCP removes useful context; the fix is safe handling, not removal.
- D. Rerunning the whole job is inefficient and still lacks explicit failure handling.

★ EXAM TIP

External/MCP calls should fail safely, visibly, and recoverably: detect, log, and fall back to a safe degraded mode.

Q79

D4 · Tool Design & MCP

You need Claude to access a live internal inventory database it otherwise cannot reach. Which extension primitive is the right choice?

- A A Skill (SKILL.md) describing the database schema.
- B An MCP server that exposes tools/resources to query the database. ✓ correct**
- C A larger CLAUDE.md documenting the tables.
- D A subagent that 'knows' the data.

WHY B IS CORRECT

MCP servers add capability — they connect Claude to systems it cannot otherwise touch, like a live database, by exposing tools/resources. That's exactly this need.

WHY THE OTHER OPTIONS ARE WRONG

- A. A skill is instructions; it can't connect to a database.
- C. CLAUDE.md documents conventions; it provides no live connection.
- D. A subagent protects context; it still needs a tool/MCP to reach the database.

★ EXAM TIP

Need to reach an external/live system? That's an MCP-shaped hole. Skills = behavior, subagents = context, MCP = capability/connection.

Q80

D4 · Tool Design & MCP

An agent has both a broad `search_everything` tool and specific `get_order` / `get_customer` tools. It frequently uses `search_everything` even for direct lookups, adding latency and noise. What is the best design change?

A Delete the specific tools and keep only `search_everything`.

B **Prefer specific, purpose-built tools for common operations and remove or narrow the broad tool so the agent chooses the right, efficient tool. ✓ correct**

C Increase the context window.

D Add more optional parameters to `search_everything`.

WHY B IS CORRECT

Purpose-built tools for common operations are more reliable and efficient than a catch-all. Narrowing or removing the broad tool nudges the agent to the correct specific tool, reducing latency and noise.

WHY THE OTHER OPTIONS ARE WRONG

A. Removing the specific tools forces everything through the noisy broad tool.

C. A bigger window doesn't fix poor tool selection.

D. More options make the broad tool even more tempting and confusing.

★ EXAM TIP

Favor specific tools for common paths; keep broad tools rare and constrained. Tool granularity guides model behavior.

Q81

D4 · Tool Design & MCP

In MCP architecture, what is the correct relationship among host, client, and server?

A The server embeds the client which embeds the host.

B **The host application contains one or more clients, and each client maintains a 1:1 connection to an MCP server that exposes tools/resources/prompts. ✓ correct**

C The client is the database and the server is the LLM.

D Host, client, and server are three names for the same process.

WHY B IS CORRECT

In MCP, the host (e.g., the AI app) runs clients; each client connects to one MCP server. Servers expose tools, resources, and prompts. This host->client->server structure standardizes external integration.

WHY THE OTHER OPTIONS ARE WRONG

- A. The nesting is reversed; the host contains clients, not the other way around.
- C. Client/server are protocol roles, not the database/LLM.
- D. They are distinct roles, not one process.

★ EXAM TIP

MCP: Host runs Clients; each Client connects 1:1 to a Server; Servers expose tools/resources/prompts.

Q82

D4 · Tool Design & MCP

A tool that creates support tickets is occasionally called twice for the same issue due to retries, creating duplicate tickets. What tool-design feature prevents this?

- A Return a bigger success payload.
- B Accept an idempotency key so repeated calls with the same key create the ticket only once. ✓ correct**
- C Add a required 'priority' field.
- D Log every call to a file.

WHY B IS CORRECT

Idempotency keys let the backend recognize duplicate calls and avoid creating multiple tickets, making retries safe for this state-changing operation.

WHY THE OTHER OPTIONS ARE WRONG

- A. Payload size is unrelated to duplication.
- C. A priority field doesn't prevent duplicates.
- D. Logging records duplicates but doesn't prevent them.

★ EXAM TIP

Any create/charge/mutate tool should support idempotency keys. It's the standard defense against duplicate side effects from retries.

Q83

D4 · Tool Design & MCP

You are designing a tool that returns a potentially huge list (e.g., all transactions). What design keeps it context-friendly and scalable?

- A Always return the entire list in one call.
- B Support pagination and filtering (limit, offset/cursor, query params) and return concise fields, so the agent fetches only what it needs. ✓ correct**
- C Return the data as one giant base64 blob.
- D Return random samples each call.

WHY B IS CORRECT

Pagination, filtering, and concise fields let the agent retrieve only relevant, bounded data, protecting the context window and improving scalability.

WHY THE OTHER OPTIONS ARE WRONG

- A. Returning everything can overflow context and waste tokens.
- C. Base64 bloats size and still must be decoded.
- D. Random sampling is unreliable and loses needed records.

★ EXAM TIP

For large result sets: paginate + filter + trim fields. Let the agent pull only what it needs, when it needs it.

Q84

D4 · Tool Design & MCP

Two tools do nearly the same thing with overlapping descriptions, and the agent keeps picking the wrong one. What is the best fix?

- A Add a third similar tool.
- B Consolidate or clearly differentiate the tools with distinct names and precise 'use when...' descriptions that state when to prefer each. ✓ correct**
- C Randomly disable one tool per session.
- D Increase temperature.

WHY B IS CORRECT

Overlapping tools confuse selection. Consolidating them or sharpening names and descriptions (explicit 'use this when...' guidance) resolves ambiguity and improves correct selection.

WHY THE OTHER OPTIONS ARE WRONG

- A. Adding another similar tool worsens the ambiguity.
- C. Randomly disabling tools is unreliable.
- D. Temperature doesn't fix overlapping semantics.

★ EXAM TIP

Avoid overlapping tools. If two tools are similar, merge them or make their names/descriptions clearly distinct with 'use when' guidance.

Q85

D4 · Tool Design & MCP

When exposing an MCP tool that performs a destructive action (e.g., `delete_records`), what is the best practice at the tool layer?

- A Trust the agent's judgment and perform the deletion immediately.
- B Add strong input validation, require explicit scoping parameters, and integrate an approval/confirmation gate so destructive actions can't fire on ambiguous or overly broad inputs. ✓ correct**
- C Return the largest possible payload after deletion.
- D Hide the tool's description so the agent rarely finds it.

WHY B IS CORRECT

Destructive tools need input validation, explicit scoping, and an approval/confirmation gate to prevent accidental or overly broad execution — enforcement at the tool layer.

WHY THE OTHER OPTIONS ARE WRONG

- A. Blind trust invites catastrophic mistakes.
- C. Payload size is irrelevant to safety.
- D. Hiding the description is security-by-obscurity, not real protection.

★ EXAM TIP

Destructive tools: validate inputs, require explicit scope, and gate with confirmation/approval. Safety belongs in the tool contract.

D5 · Context & Reliability

Context Management & Reliability · Exam weight 15% · 15 questions

Q86**D5 · Context & Reliability**

A vendor changes an invoice layout and a previously reliable field is now missing in 30% of extractions, with no schema change. What monitoring detects such 'extraction drift' earliest?

- A** Re-run extraction on a sample once a month.
- B** Wait for downstream consumers to complain.
- C** Manually audit a random sample weekly.
- D** Track per-field non-null (population) rates over time and alert on significant drops (e.g., 96% last week vs. 67% this week). ✓ correct

WHY D IS CORRECT

Per-field population/non-null rate monitoring with alerting on sudden drops catches silent extraction degradation quickly and automatically, before consumers notice.

WHY THE OTHER OPTIONS ARE WRONG

- A.** Monthly sampling is far too infrequent; issues persist for weeks.
- B.** Waiting for complaints is reactive; data quality already suffered.
- C.** Manual weekly audits don't scale and may miss vendor-specific drops.

★ EXAM TIP

Monitor per-field non-null rates (and validation failure rates) over time with alerts. Sudden drops = drift signal for early detection.

Q87**D5 · Context & Reliability**

Extraction records that fail validation are silently dropped into a quarantine queue and never analyzed. What practice best improves the system over time?

- A** Ignore rejected records; consumers can cope.
- B** Build a feedback loop: periodically review sampled rejections (with failure reasons) and use recurring patterns to improve prompts, schemas, preprocessing, and routing — treating validation as a quality signal, not just a gate. ✓ correct
- C** Lower the validation threshold so fewer records fail.
- D** Manually re-extract each failure one by one.

WHY B IS CORRECT

Validation failures are valuable feedback. Reviewing sampled rejections and their reasons reveals recurring patterns that drive systematic improvements (prompt/schema/preprocessing/routing), steadily reducing rejections.

WHY THE OTHER OPTIONS ARE WRONG

- A. Ignoring failures guarantees the issues persist.
- C. Lowering the threshold hides problems and degrades quality.
- D. One-by-one re-extraction fixes instances but not systemic causes.

★ EXAM TIP

Make validation a feedback source, not just a gate. Mine failure patterns to improve the pipeline continuously.

Q88

D5 · Context & Reliability

Extraction quality drops when email bodies include signatures, disclaimers, and quoted reply chains that leak into fields like 'notes.' What is the cleanest fix?

- A Trust the model to ignore signatures and quoted text.
- B Extract from the full email, then post-filter mistakes.
- C Pre-process emails to isolate the latest message body, strip signatures and quoted replies before extraction, and optionally scope the schema to the current message body only. ✓ correct**
- D Refuse to process emails.

WHY C IS CORRECT

Cleaning the input first — isolating the latest message and removing signatures/quoted history — prevents leakage far more reliably than trusting the model or cleaning up afterward.

WHY THE OTHER OPTIONS ARE WRONG

- A. Models aren't perfect; signatures/quotes often contain names, emails, and company info that get mis-extracted.
- B. Post-filtering lets errors happen first and is more complex.
- D. Emails are a valid source; the fix is preprocessing, not refusal.

★ EXAM TIP

Clean inputs before extraction: isolate the latest message, strip signatures/quotes. Upstream preprocessing beats downstream cleanup.

Q89

D5 · Context & Reliability

You need Claude Code to understand a 3,000-file codebase to trace how authentication works. What exploration strategy is most effective?

- A Read documentation from external sites only.
- B Read every file in the repository.
- C Read only the application entry point.
- D Use Grep and Glob to search for auth-related patterns (authenticate, JWT, session, login), then load only the identified relevant files for deeper analysis. ✓ correct

WHY D IS CORRECT

For large codebases, search first to narrow the space (Grep/Glob for keywords and file patterns), then deeply analyze the small relevant subset. This is efficient and scalable.

WHY THE OTHER OPTIONS ARE WRONG

- A. Docs may be stale and incomplete; the question asks about codebase exploration.
- B. Reading 3,000 files is infeasible and wasteful.
- C. Auth is usually spread across middleware, services, and models — the entry point alone is insufficient.

★ EXAM TIP

Large codebase strategy: search (Grep/Glob) to find the relevant ~1%, then read those files. Don't brute-force read everything.

Q90

D5 · Context & Reliability

A developer asks Claude Code 'How does payment processing work?' It jumps straight into `src/payments/stripe.ts` and misses the overall design. What is the better exploration approach?

- A Start with high-level files (README.md, CLAUDE.md, package.json dependencies, the `/src/payments/` directory structure, architecture docs), then drill into `stripe.ts` with that context. ✓ correct
- B Ask the developer which files to read.
- C Read every file alphabetically.
- D Read the external Stripe API docs first.

WHY A IS CORRECT

Architecture questions call for top-down exploration: understand structure, dependencies, and design docs before diving into a single implementation file. That context makes `stripe.ts` meaningful.

WHY THE OTHER OPTIONS ARE WRONG

- B. Exploration is Claude Code's job here; the developer is asking it to figure this out.
- C. Alphabetical reading is inefficient and meaningless.
- D. The question is about THIS project's architecture, not Stripe itself.

★ EXAM TIP

For 'how does X work' questions, go top-down: docs -> structure -> dependencies -> key interfaces -> implementation. Broad before deep.

Q91

D5 · Context & Reliability

To understand a large monorepo with 5 major modules where sequential exploration risks context exhaustion, what pattern works best?

- A Read only the top-level directory listing and infer architecture from folder names.
- B Use the Task tool to spawn parallel subagent explorations (one per module), each returning a structured summary, then have the coordinator synthesize them into an overall architecture understanding. ✓ correct**
- C Load all 5 modules into context simultaneously and ask for a summary.
- D Explore only the most important module and assume the rest follow the same patterns.

WHY B IS CORRECT

Parallel subagents, each exploring one module in an isolated context and returning a summary, avoid context exhaustion and scale well. The coordinator then synthesizes the summaries.

WHY THE OTHER OPTIONS ARE WRONG

- A. Folder names are too shallow to reveal real architecture.
- C. Loading everything at once causes the exact context exhaustion you're avoiding.
- D. Modules often differ; assuming uniformity yields wrong conclusions.

★ EXAM TIP

Large multi-module exploration: partition -> parallel subagents -> summaries -> synthesize. Subagents keep each slice out of the parent context.

Q92

D5 · Context & Reliability

A system prompt plus a large, stable reference document is sent on thousands of requests, and costs/latency are high. Which feature most directly reduces repeated processing cost?

- A Lower the temperature.
- B Enable prompt caching for the stable prefix (system prompt + reference document) so repeated tokens are reused across requests at reduced cost/latency. ✓ correct**
- C Switch every request to a larger model.
- D Remove the reference document entirely.

WHY B IS CORRECT

Prompt caching stores a stable prefix so repeated large context (system prompt + reference doc) isn't reprocessed from scratch each call, cutting cost and latency for high-volume, repeated-context workloads.

WHY THE OTHER OPTIONS ARE WRONG

- A. Temperature affects randomness, not repeated-processing cost.
- C. A larger model raises cost; it doesn't cache repeated context.
- D. Removing the document may break the task; caching keeps it cheaply.

★ EXAM TIP

Stable, reused context + high volume = prompt caching. Cache the unchanging prefix (system + reference) to slash repeated cost/latency.

Q93

D5 · Context & Reliability

A knowledge assistant must answer from a corpus far larger than any context window. Which architecture is most appropriate?

- A Paste the entire corpus into every prompt.
- B Use Retrieval-Augmented Generation (RAG): index the corpus, retrieve only the most relevant chunks per query, and provide those to Claude as context. ✓ correct**
- C Fine-tune the model on every document nightly.
- D Summarize the whole corpus into one paragraph and rely on that.

WHY B IS CORRECT

RAG retrieves only the most relevant chunks per query and injects them as context, letting Claude answer from a corpus far larger than the context window without pasting everything.

WHY THE OTHER OPTIONS ARE WRONG

- A. The corpus exceeds the context window; pasting it all is impossible/wasteful.
- C. Nightly fine-tuning is expensive, slow, and unnecessary for retrieval-style QA.
- D. A single paragraph loses the detail needed to answer specific questions.

★ EXAM TIP

Corpus bigger than the window? Use RAG: retrieve relevant chunks per query instead of stuffing everything into context.

Q94

D5 · Context & Reliability

A very long-running chat is approaching the context window limit and older but still-relevant details risk being lost. What is a sound reliability strategy?

- A Do nothing and let the model forget.
- B Periodically summarize/compact earlier turns into a compact running summary retained in context, keeping key facts while freeing space. ✓ correct**
- C Randomly delete half the messages.
- D Switch to temperature 0.

WHY B IS CORRECT

Summarizing/compacting older turns into a running summary preserves the important facts while freeing context space, keeping long conversations coherent and within limits.

WHY THE OTHER OPTIONS ARE WRONG

- A. Doing nothing loses important context and degrades quality.
- C. Random deletion may drop critical information.
- D. Temperature doesn't manage context length.

★ EXAM TIP

Manage long conversations by compacting older turns into a running summary. Preserve key facts, free space, maintain coherence.

Q95

D5 · Context & Reliability

Which practice most improves reliability when calling the Claude API in production against transient failures (timeouts, 429 rate limits, 5xx)?

- A Fail immediately on the first error.
- B Retry transient errors with exponential backoff and jitter, respect rate-limit headers, and cap total attempts before surfacing a clear error. ✓ correct**
- C Retry instantly and infinitely with no delay.
- D Switch models on every error.

WHY B IS CORRECT

Exponential backoff with jitter, respecting rate-limit signals, and a capped retry count is the standard resilient pattern for transient API failures — recovering from blips without hammering the service.

WHY THE OTHER OPTIONS ARE WRONG

- A. Failing on the first transient error wastes recoverable situations.
- C. Infinite no-delay retries cause thundering-herd and worsen rate limiting.
- D. Swapping models per error doesn't address transient failures and adds complexity.

★ EXAM TIP

Resilient API calls: retry transient errors with exponential backoff + jitter, honor rate limits, cap attempts. Don't retry non-transient errors.

Q96

D5 · Context & Reliability

When choosing a production model, a high-volume classification task needs low latency and low cost with 'good enough' accuracy. Which anchoring choice is most appropriate?

- A Use the largest, most capable model for everything.
- B Choose a smaller, faster model (e.g., a Haiku-tier model) for the high-volume, well-scoped task, reserving larger models for genuinely hard reasoning. ✓ correct**
- C Always pick the newest model regardless of fit.
- D Randomly alternate models to balance load.

WHY B IS CORRECT

Model selection is a cost/latency/quality trade-off. For high-volume, well-scoped tasks, a smaller/faster tier gives low latency and cost with sufficient accuracy; save larger models for hard reasoning.

WHY THE OTHER OPTIONS ARE WRONG

- A. Using the biggest model everywhere wastes cost and latency.
- C. Newest isn't automatically the right fit; the current blueprint anchors to production-tier models.
- D. Random alternation makes performance unpredictable.

★ EXAM TIP

Match model tier to task: small/fast (Haiku-tier) for high-volume simple tasks; larger tiers for hard reasoning. Optimize cost/latency/quality, not raw size.

Q97

D5 · Context & Reliability

A production extraction pipeline occasionally returns JSON that fails to parse, breaking downstream systems. What reliability practice best prevents bad data from propagating?

- A Assume Claude always returns valid JSON and skip checks.
- B Validate every response against the schema; on failure, retry with a corrective prompt or route to a quarantine/flag path instead of passing invalid data downstream. ✓ correct**
- C Wrap the output in try/except and silently drop failures.
- D Increase temperature to add variety.

WHY B IS CORRECT

Schema validation at the boundary, with retry-on-failure or quarantine/flagging, stops malformed output from reaching downstream systems and provides a recovery path.

WHY THE OTHER OPTIONS ARE WRONG

- A. Assuming valid JSON guarantees eventual breakage.
- C. Silently dropping failures hides data loss and problems.
- D. Higher temperature increases malformed-output risk.

★ EXAM TIP

Validate at the boundary; on failure retry or quarantine. Never trust unvalidated model output in production pipelines.

Q98

D5 · Context & Reliability

You want to reduce cost and latency for repeated tool-augmented conversations that reuse the same large system prompt and tool definitions. Which combination helps most?

- A Remove all tool definitions.
- B Cache the stable prefix (system prompt + tool definitions) with prompt caching, and keep per-request dynamic content minimal and after the cached prefix. ✓ correct**
- C Send the tool definitions twice for safety.
- D Use the largest model to 'get it right the first time.'

WHY B IS CORRECT

Placing stable content (system prompt + tool definitions) in a cached prefix and keeping dynamic content minimal and after it maximizes cache hits, cutting cost and latency across repeated calls.

WHY THE OTHER OPTIONS ARE WRONG

- A. Removing tools breaks functionality.
- C. Duplicating definitions wastes tokens and can confuse the model.
- D. A bigger model increases cost without caching benefits.

★ EXAM TIP

Structure prompts as [stable cached prefix] + [small dynamic suffix]. Cache the unchanging system/tool content to boost cache hits.

Q99

D5 · Context & Reliability

An agent's context is filling with verbose raw tool outputs from many calls, crowding out earlier important information. What is the best mitigation?

- A Let the context fill and hope the model prioritizes correctly.
- B Have tools return concise, curated results and/or delegate noisy work to subagents that return only summaries, keeping the parent context lean. ✓ correct**
- C Duplicate the important information at the end of every message.
- D Increase temperature.

WHY B IS CORRECT

Curating tool outputs to essentials and delegating noisy tasks to subagents (which return summaries) keeps verbose intermediate data out of the parent context, protecting important information and the window.

WHY THE OTHER OPTIONS ARE WRONG

- A. Hoping the model prioritizes correctly under overflow is unreliable.
- C. Duplication consumes even more context.
- D. Temperature doesn't manage context volume.

★ EXAM TIP

Protect the parent context: return concise tool results and push noisy work to subagents that return summaries. Keep the main window lean.

Q100

D5 · Context & Reliability

For a production Claude application, which observability practice most improves reliability and faster incident response?

- A Log nothing to protect privacy absolutely.
- B Track key metrics (latency, error/refusal rates, token usage, validation failure rates, tool-call success) and log requests/responses with appropriate redaction, so regressions and drift are detected quickly. ✓ correct**
- C Only measure total monthly spend.
- D Rely on users to report every problem.

WHY B IS CORRECT

Comprehensive metrics (latency, errors, tokens, validation failures, tool success) plus redacted logging enable fast detection of regressions, drift, and incidents — the foundation of reliable operations.

WHY THE OTHER OPTIONS ARE WRONG

- A. Logging nothing leaves you blind; use redaction instead of no logging.
- C. Monthly spend alone misses quality and reliability signals.
- D. User reports are lagging and incomplete.

★ EXAM TIP

Instrument production: latency, error/refusal rates, tokens, validation failures, tool success. Add redacted logging. You can't fix what you don't measure.